



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE MADRID

**ESCUELA DE ARQUITECTURA, INGENIERÍA Y
DISEÑO**

GRADO EN FÍSICA

TRABAJO FIN DE GRADO

**APLICACIÓN DE TENSOR
NETWORKS COMO MODELOS DE
DEEP LEARNING PARA
CLASIFICACIÓN DE IMÁGENES**

MANUEL ARENAS SÁNCHEZ

Dirigido por

ALEJANDRO MATA ALI

MARÍA FUENCISLA GILSANZ MUÑOZ

CURSO 2025-2026

TÍTULO: APLICACIÓN DE TENSOR NETWORKS COMO MODELOS DE DEEP LEARNING PARA CLASIFICACIÓN DE IMÁGENES

AUTOR: MANUEL ARENAS SÁNCHEZ

TITULACIÓN: GRADO EN FÍSICA

DIRECTOR/ES DEL PROYECTO: ALEJANDRO MATA ALI
MARÍA FUENCISLA GILSANZ MUÑOZ

FECHA: MARZO DE 2026

RESUMEN

Este Trabajo de Fin de Grado estudia la aplicación de las *Tensor Networks* como modelos de *deep learning* para tareas de clasificación de imágenes. El objetivo principal es analizar si este tipo de arquitecturas, procedentes originalmente de la física cuántica y del álgebra multilineal, pueden integrarse de forma práctica en un flujo moderno de *machine learning*. Para ello se revisan los fundamentos teóricos de las redes tensoriales, las principales topologías empleadas en la literatura y su relación con modelos entrenables mediante diferenciación automática.

La parte práctica del proyecto consiste en el diseño e implementación de un *pipeline* completo de clasificación en PyTorch, apoyado en TensorKrowch para las contracciones tensoriales. El sistema desarrollado incluye la preparación de datos, la generación de recortes a partir de anotaciones YOLO, el preprocesamiento de imágenes, la definición de modelos mediante archivos de configuración, el entrenamiento, la evaluación y el registro reproducible de experimentos. El trabajo utiliza el dataset ChemEq25, un conjunto extenso de imágenes de material de laboratorio.

La arquitectura final combina un **AIMPatchEmbedding**, encargado de extraer una representación visual inicial, con una **HybridCustomTTN** basada en bloques tensoriales jerárquicos. A lo largo del proyecto se evaluaron distintas decisiones de diseño, como el tipo de *embedding*, la dimensión interna del modelo, el *learning rate*, la regularización, el *label smoothing*, la preservación de la relación de aspecto y el aumento de datos. Los resultados muestran una mejora progresiva desde los primeros experimentos hasta el mejor modelo obtenido, que alcanzó un 96,05 % de *accuracy*, un 95,94 % de F1 macro y un 99,43 % de *top-5 accuracy* sobre ChemEq25.

El trabajo concluye que las *Tensor Networks* pueden utilizarse de forma viable en clasificación de imágenes cuando se integran dentro de un sistema de entrenamiento y preprocesamiento adecuado. No obstante, también se observa que el rendimiento no depende únicamente de la topología tensorial, sino de la representación de entrada, la regularización, el aumento de datos y la trazabilidad experimental. Por tanto, el proyecto aporta una implementación funcional y un análisis crítico de las ventajas y limitaciones de estos modelos en un problema de clasificación de imágenes.

ABSTRACT

This Bachelor's Thesis studies the application of *Tensor Networks* as deep learning models for image classification tasks. The main objective is to analyse whether this type of architecture, originally developed in quantum physics and multilinear algebra, can be practically integrated into a modern machine learning workflow. To this end, the theoretical foundations of tensor networks, the main topologies used in the literature and their relation to trainable models based on automatic differentiation are reviewed.

The practical part of the project consists of the design and implementation of a complete image classification pipeline in PyTorch, supported by TensorKrowch for tensor contractions. The developed system includes data preparation, crop generation from YOLO annotations, image preprocessing, model definition through configuration files, training, evaluation and reproducible experiment logging. This project use the dataset ChemEq25, a dataset of laboratory equipment.

The final architecture combines an `AIMPatchEmbedding`, responsible for extracting an initial visual representation, with a `HybridCustomTTN` based on hierarchical tensor blocks. Throughout the project, different design decisions were evaluated, including the type of embedding, the internal model dimension, the learning rate, regularisation, label smoothing, aspect-ratio preservation and data augmentation. The results show a progressive improvement from the first experiments to the best model obtained, which achieved 96.05 % accuracy, 95.94 % macro F1 and 99.43 % top-5 accuracy on ChemEq25.

The thesis concludes that *Tensor Networks* can be viably used for image classification when they are integrated into an appropriate training and preprocessing system. However, the results also show that performance does not depend only on the tensor topology, but also on the input representation, regularisation, data augmentation and experimental traceability. Therefore, the project provides a functional implementation and a critical analysis of the advantages and limitations of these models in a real visual problem.

Keywords: Tensor Networks, deep learning, image classification, PyTorch, TensorKrowch, Tree Tensor Network, ChemEq25.

AGRADECIMIENTOS

En primer lugar, me gustaría dar las gracias a Alejandro Mata Ali por toda la ayuda que me ha prestado. Ha seguido mi progreso de cerca en todo momento, me ha facilitado una gran cantidad de material de estudio y me ha ofrecido nuevas perspectivas siempre que me he atascado en algún problema.

Quiero agradecer también a María Fuencisla Gilsanz Muñoz, mi tutora en la UEM, que igualmente ha acompañado mi trabajo en todo momento y me ha aportado numerosas ideas y mucho material durante la realización de este TFG.

También quiero agradecer a todas las personas con las que he trabajado en el Instituto Tecnológico de Castilla y León (ITCL), y en especial al departamento de computación cuántica, formado por Sergio Muñiz Subiñas, Miguel Franco Hernando, Jorge Martínez Martín y Alejandro Mata Ali. Ellos han sido una pieza fundamental, pues gracias a su acompañamiento he aprendido y disfrutado enormemente de mis prácticas en el ITCL, además de recibir muchos consejos sobre el desarrollo de este proyecto. Sin ellos, nada de esto habría sido lo mismo.

Quisiera dedicar una mención muy especial al profesor Iñaki Ortiz Delandaluce, que siempre ha creído en mí y cuyo apoyo incondicional ha supuesto para mí una motivación enorme. Sin él, probablemente no habría entrado en el ITCL y, muy posiblemente, este TFG no existiría.

Por último, quiero agradecer a las personas más importantes para mí: mi familia; que me han apoyado siempre a lo largo de toda la carrera de física, en la que hemos atravesado momentos muy difíciles. Siempre creyeron en mí y me dieron fuerzas hasta el final.

Cita - frase célebre / Dedicatoria

“It is not our part to master all the tides of the world, but to do what is in us for the succour of those years wherein we are set, uprooting the evil in the fields that we know, so that those who live after may have clean earth to till. What weather they shall have is not ours to rule.”

J.R.R. Tolkien

TABLA RESUMEN

	DATOS
Nombre y apellidos:	Manuel Arenas Sánchez
Título del proyecto:	Aplicacion de Tensor Networks en modelos de IA para clasificación de imágenes. . .
Directores del proyecto:	Alejandro Mata Ali y Maria Fuencisla Gilsanz
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa:	SI
El proyecto ha implementado un producto: (esta entrada se puede marcar junto a la siguiente)	NO
El proyecto ha consistido en el desarrollo de una investigación o innovación: (esta entrada se puede marcar junto a la anterior)	SI
Objetivo general del proyecto:	Estudiar la implementación de las <i>tensor networks</i> en <i>deep learning</i>

ÍNDICE

RESUMEN	4
ABSTRACT	5
AGRADECIMIENTOS	6
TABLA RESUMEN	8
1 INTRODUCCIÓN	12
1.1 Estado del Arte	12
1.2 Planteamiento del Problema	14
1.3 Planificación del TFG	16
1.4 Recursos requeridos	18
2 OBJETIVOS	19
2.1 Objetivos generales	19
2.2 Objetivos específicos	19
2.3 Objetivos que quedan fuera del proyecto	20
2.4 Beneficios del proyecto	20
3 MARCO TEÓRICO	22
3.1 Deep learning y clasificación de imágenes	22
3.2 Tensores	24
3.3 Notación diagramática de Penrose y definición de Tensor Network	26
3.4 Tensor Networks y reducción de dimensionalidad	28
3.5 Descomposiciones tensoriales fundamentales	29
3.6 Topologías principales de Tensor Networks	31
3.6.1 Matrix Product State / Tensor Train	31
3.6.2 Tree Tensor Network	32

3.6.3	MERA	33
3.6.4	PEPS	33
4	METODOLOGÍA	35
4.1	Pipeline computacional de clasificación en PyTorch	35
4.2	Organización del proyecto	36
4.3	Preparación del dataset ChemEq25	37
4.4	Transformaciones y aumento de datos	38
4.5	Embeddings	38
4.6	Arquitectura tensorial	39
4.7	Entrenamiento y regularización	40
4.8	Métricas, diagnósticos y trazabilidad	40
4.9	Ejecución en servidor	41
5	RESULTADOS	42
5.1	Primeros experimentos con Flowers102	42
5.1.1	Modelo original CustomTTN	42
5.1.2	Hipótesis 1	43
5.1.3	Modelo HybridCustomTTN	43
5.1.4	Hipótesis 2	45
5.2	Cambio del embedding a AIM	45
5.2.1	Hipótesis 3	48
5.3	Cambio de dataset a ChemEq25	48
5.3.1	Efecto del learning rate	50
5.4	Ajustes de entrenamiento y regularización	51
5.4.1	Hipótesis 4	54
5.5	Salto con data augmentation light	54
5.6	Mejor modelo obtenido	55
5.7	Variabilidad por seed y límite de las pruebas finales	56
5.8	Resumen de la evolución experimental	58
6	CONCLUSIONES	59

6.1 Conclusiones del trabajo	59
6.2 Conclusiones personales	60
REFERENCIAS	63

ÍNDICE DE FIGURAS

1.1	Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.	16
3.1	Esquema de una red neuronal simple. [1]	22
3.2	Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].	24
3.3	Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia usando la librería Tensor-Network-Editor, creada por Alejandro Mata Ali. [3]	27
3.4	Ejemplo de representación diagramática para un tensor. Fuente: [4]	28
3.5	Diagramas TN de algunas descomposiciones TN populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (c) La descomposición en términos de bloque descompone un tensor X en una suma de varias descomposiciones de Tucker, con rangos de Tucker bajos. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (e) La descomposición TR descompone un tensor X en un conjunto de tensores núcleo de tercer orden y los contrae en una estructura de anillo. (f) La descomposición HT representa un tensor X como un diagrama con estructura de árbol. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. [5]	31
4.1	Pipeline completo del modelo propuesto	36

4.2	Diagrama del la TTN. Es una imagen orientativa, ya que en ella se representan 3 capas cuando la TTN original tiene 5 capas. Esta imagen es de elaboración propia, usando la librería Tensor-Network-Editor, creada por Alejandro Mata Ali. [3]	39
4.3	Captura de una GPU NVIDIA RTX 4000 Ada Generation del servidor Lorca de la UEM ejecutando dos procesos de entrenamiento en paralelo.	41
5.1	Curvas de pérdida del primer experimento con CustomTTN sobre Flowers102.	42
5.2	Primer experimento con la arquitectura híbrida. La pérdida ya no queda tan cerca del comportamiento aleatorio inicial.	44
5.3	Experimento largo con HybridCustomTTN sobre Flowers102.	44
5.4	Experimento con AIMPatchEmbedding durante 50 épocas.	46
5.5	Experimento con AIMPatchEmbedding durante 100 épocas.	46
5.6	Experimentos en Flowers102 con AIM y <i>data augmentation</i>	47
5.7	Matrices de confusión multiclase en Flowers102 tras AIM y <i>data augmentation</i>	48
5.8	En la figura a) vemos ejemplos de cajas obtenidas a partir de las anotaciones YOLO. En la figura b) vemos ejemplos de los recortes utilizados como entrada del clasificador. El color gris que se ven entre las imágenes es el color de relleno cercano a la media de <i>ImageNet</i> . Las imágenes en b) son los recortes tal y como las va a ver el modelo.	49
5.9	Primer experimento con ChemEq25.	49
5.10	Mismo experimento que Figura 5.9 , pero con <i>learning rate</i> menor.	50
5.11	Distintos experimentos a los que se le fueron aplicando pequeñas mejoras detalladas en Tabla 5.9 . Se puede ver como las curvas de perdida van mejorando con cada experimento.	52
5.12	Matrices de confusión de distintos experimentos a los que se le fueron aplicando pequeñas mejoras detalladas en Tabla 5.9 . Se puede ver como la diagonal se hace más evidente a medida que se aplican los cambios.	53
5.13	Experimentos realizados con el cambio del <i>data augmentation</i>	55
5.14	Matrices de confusión de los experimentos después de aplicar <i>data augmentation</i>	57

ÍNDICE DE TABLAS

3.1	Comparación entre las descomposiciones CP, Tucker y TT/MPS.	30
3.2	Comparativa sintética de topologías relevantes de Tensor Networks.	34
5.1	Resumen representativo del primer experimento con <code>CustomTTN</code>	43
5.2	Resumen del primer experimento relevante con <code>HybridCustomTTN</code>	44
5.3	Resumen del experimento largo con <code>HybridCustomTTN</code>	44
5.4	Primer resultado con <code>AIMPatchEmbedding</code>	46
5.5	Resultado con AIM durante 100 épocas.	46
5.6	Mejores resultados sobre Flowers102 después de añadir AIM y <i>data augmentation</i>	47
5.7	Primer resultado sobre ChemEq25.	50
5.8	Comparación de los primeros ajustes sobre ChemEq25.	50
5.9	Evolución de los ajustes de entrenamiento antes del último cambio de <i>data augmentation</i>	51
5.10	Resultados con <code>augmentation_name = light</code>	54
5.11	Métricas principales del mejor modelo sobre ChemEq25.	56
5.12	Seed sweep posterior al mejor resultado.	56
5.13	Resumen global de la evolución experimental del proyecto.	58

Capítulo 1. INTRODUCCIÓN

La clasificación de imágenes es una de las tareas centrales del *machine learning*. Consiste en asignar a una imagen una etiqueta entre un conjunto finito de clases, a partir de patrones visuales que pueden depender, de forma simultánea, del color, la forma, la textura, la escala, la iluminación y el contexto. En los últimos años, los modelos de *deep learning* han demostrado una gran capacidad para resolver este tipo de problemas, pero también han puesto de manifiesto un problema importante, los modelos más expresivos suelen requerir un número elevado de parámetros, grandes cantidades de datos y un coste computacional considerable.

Este TFG parte de esa problemática y estudia una familia de modelos alternativa, inspirada en herramientas procedentes de la física cuántica y del álgebra multilineal; las Tensor Networks (TN). La idea general es representar objetos de dimensión muy alta mediante la contracción de tensores más pequeños, de forma que el número de parámetros crezca de manera controlada.

La motivación física aparece de manera natural. En mecánica cuántica, describir exactamente un sistema de muchas partículas exige manejar espacios de Hilbert cuya dimensión crece exponencialmente con el número de componentes. Las TN surgieron precisamente de una idea que sirvió para representar de forma eficiente estados cuánticos relevantes sin almacenar explícitamente todos sus coeficientes. En *machine learning* aparece un problema análogo. Una imagen contiene muchas variables, y una función de decisión general sobre todas ellas vive, en principio, en un espacio de dimensión enorme. La pregunta que guía este trabajo es, si las estructuras tensoriales pueden aprovechar regularidades de los datos visuales, para construir clasificadores entrenables y analizables.

1.1 Estado del Arte

La literatura reciente sobre las Tensor Networks aplicadas a *machine learning* puede organizarse en tres líneas principales. La primera utiliza descomposiciones tensoriales para reducir dimensionalidad o comprimir modelos ya existentes. En esta línea, las TN se emplean como herramienta de factorización de pesos, capas densas, convoluciones o representaciones intermedias, con el objetivo de reducir memoria y coste computacional [6, 5]. La segunda línea utiliza las TN como modelos de aprendizaje en sí mismos, es decir, como clasificadores o generadores entrenables. La tercera, más reciente, estudia arquitecturas híbridas que combinan ideas tensoriales con mecanismos habituales de *deep learning*, como normalización, *patch embeddings*, capas convolucionales ligeras o entrenamiento extremo a extremo con *backpropagation* [5].

Un punto de partida fundamental para la aplicación de TN a clasificación supervisada

es el trabajo de Stoudenmire y Schwab [7]. En dicho trabajo, una imagen se transforma mediante un mapa local de características y el clasificador se interpreta como una función lineal en un espacio de características de dimensión exponencial, cuyo tensor de pesos se representa de forma compacta mediante una Matrix Product State (MPS). El resultado mostró que una MPS podía alcanzar un error de test inferior al 1% en MNIST. Este resultado fue importante porque demostró que las redes tensoriales podían actuar como modelos supervisados, no solo como herramientas en física cuántica o como herramientas de compresión. Sin embargo, también marcó una limitación que sigue siendo relevante; la MPS impone una estructura lineal sobre los datos. En imágenes sencillas y pequeñas, como MNIST, esta linealización puede ser suficiente; sin embargo, en imágenes naturales, en color o de mayor resolución, la pérdida de localidad espacial se vuelve mucho más problemática.

Una respuesta natural a esa limitación consiste en utilizar topologías que respeten mejor la estructura bidimensional de las imágenes. Los Projected Entangled Pair States (PEPS) son especialmente atractivos porque organizan los tensores sobre una rejilla 2D, más cercana a la geometría original de una imagen. Cheng, Wang y Zhang aplicaron PEPS a clasificación supervisada en MNIST y Fashion-MNIST, mostrando que una red tensorial bidimensional puede superar a topologías tipo árbol en esos escenarios y funcionar con un número reducido de parámetros [8]. No obstante, PEPS introduce una dificultad importante; la contracción exacta de redes tensoriales bidimensionales es computacionalmente intratable en el caso general [9]. Por ello, aunque PEPS es conceptualmente muy adecuado para imágenes, su uso práctico suele requerir contracciones aproximadas, restricciones en las dimensiones de enlace o implementaciones cuidadosas.

Entre la MPS lineal y PEPS bidimensional aparecen las Tree Tensor Networks (TTN), que organizan la información de manera jerárquica. Esta familia resulta interesante porque reduce la distancia efectiva entre regiones alejadas de la entrada y permite agregar información de forma progresiva. En comparación con una cadena MPS, una TTN puede capturar interacciones de mayor alcance con menos pasos de propagación. Aun así, al no tener ciclos, las interacciones entre ramas distintas quedan mediadas por ancestros comunes, lo que limita la comunicación directa entre regiones separadas. Por tanto, la TTN representa un compromiso entre eficiencia de contracción, estructura jerárquica y capacidad expresiva [5, 10].

La tendencia más reciente no se limita a elegir una topología física clásica, sino a re-interpretar las TN desde el punto de vista de las arquitecturas modernas. Un ejemplo especialmente relevante es Deep Tree Tensor Networks (DTTN), que propone apilar módulos de interacción antisimétrica (AIM) para capturar interacciones multiplicativas de orden creciente y mantener, al mismo tiempo, una estructura desplegable como red tensorial de tipo árbol [11]. Este trabajo es importante para el presente TFG por dos motivos. Primero, identifica de forma explícita que las MPS tradicionales han tenido dificultades para escalar a reconocimiento de imágenes naturales cuando se usan como núcleo principal del modelo. Segundo, conecta las TN con redes polinómicas y multilineales modernas,

mostrando que la investigación actual se mueve hacia arquitecturas híbridas, entrenables con herramientas estándar y mejor adaptadas a datos visuales complejos.

Desde el punto de vista de implementación, también ha habido una evolución clara. Las primeras propuestas de aprendizaje con MPS estaban muy ligadas a algoritmos inspirados en DMRG y a código de investigación específico. Actualmente existen librerías que buscan integrar TN dentro de ecosistemas de *deep learning*. TensorKrowch, por ejemplo, está construido sobre PyTorch y permite definir redes tensoriales entrenables, conectarlas con otros módulos y aprovechar diferenciación automática [12]. Esta disponibilidad de herramientas no elimina las dificultades del problema; siguen siendo decisiones críticas la elección del *embedding*, el orden de las posiciones de entrada de datos, la topología, la dimensión de enlace, la ruta de contracción y la estabilidad del entrenamiento.

En conjunto, el estado del arte muestra que las TN son una línea prometedora para construir modelos compactos, estructurados e interpretables, pero no constituyen todavía una alternativa universal a las arquitecturas dominantes de *machine learning*, como CNNs o Transformers. Su ventaja es más clara cuando se busca controlar el número de parámetros, introducir sesgos estructurales, modelar interacciones multilineales o estudiar arquitecturas inspiradas en física cuántica. Su mayor debilidad práctica aparece al trasladar resultados de datasets sencillos a imágenes reales de mayor resolución, donde la representación espacial y el coste de contracción se vuelven determinantes. Este TFG se sitúa precisamente en ese punto. Parte de modelos consolidados como MPS, estudia alternativas jerárquicas tipo TTN y explora mecanismos híbridos inspirados en trabajos recientes, sin asumir que una red tensorial vaya a superar de forma automática a una *deep learning* convencional.

1.2 Planteamiento del Problema

El problema que aborda este TFG puede formularse de la siguiente manera. Estudiar si una arquitectura basada en Tensor Networks puede utilizarse de forma práctica para resolver una tarea de clasificación de imágenes, manteniendo un número controlado de parámetros y una estructura interpretable, sin depender exclusivamente de arquitecturas profundas convencionales.

Esta formulación general contiene varios retos. El primero es la representación de la imagen. Una imagen digital no es, para una red tensorial, una entrada natural en forma de matriz o mapa de píxeles, sino un objeto que debe convertirse previamente en una secuencia de vectores locales. En datasets simples como el MNIST, esta conversión puede hacerse píxel a píxel. Sin embargo, cuando las imágenes son RGB, tienen mayor resolución y contienen objetos reales con forma, orientación y textura, tratar cada píxel como un sitio tensorial produce secuencias demasiado largas y poco manejables. Por tanto, una parte central del problema consiste en diseñar un *embedding* que reduzca la imagen a una representación compatible con la TN, pero que no destruya por completo la información espacial relevante.

El segundo problema es la elección de la topología tensorial. Una MPS resulta atractiva por su simplicidad y por la madurez de sus algoritmos, pero impone una geometría lineal que no se ajusta de forma natural a una imagen bidimensional. Una PEPS conserva mejor la estructura 2D, pero su contracción exacta es mucho más costosa. Una TTN ofrece una solución intermedia; organiza la información de forma jerárquica, reduce la distancia efectiva entre posiciones de entrada y mantiene una contracción más manejable. Por ello, el proyecto plantea estudiar arquitecturas tensoriales que puedan entrenarse en la práctica y que sean razonables para imágenes reales, aunque no reproduzcan literalmente todos los modelos más complejos de la literatura.

El tercer problema es de implementación. Pasar de una definición matemática de una Tensor Network a un modelo entrenable requiere un nivel de computación elevado. Es necesario construir nodos tensoriales con parámetros, conectar correctamente sus índices, definir una ruta de contracción, integrar la red con PyTorch, calcular gradientes, entrenar por *batches* y registrar métricas. Además, las capas tensoriales deben encajar con el resto del sistema de clasificación, carga de datos, transformaciones, función de pérdida, optimizador, *scheduler*, *checkpoints* y evaluación.

El cuarto problema es experimental. No basta con que el modelo pueda ejecutarse y disminuir la pérdida de entrenamiento; hay que evaluar si aprende patrones visuales que generalicen al conjunto de validación y test. Para ello, el trabajo no puede limitarse a una única métrica final. Es necesario analizar precisión, *macro F1*, curvas de entrenamiento, matrices de confusión, comportamiento por clase y posibles señales de *overfitting* o sobreconfianza. Esta evaluación es especialmente importante porque las TN pueden ser eficientes en parámetros, pero esa eficiencia no garantiza por sí misma un buen rendimiento en clasificación de imágenes.

En la versión final del proyecto, el problema se concreta sobre ChemEq25, un dataset de imágenes de material de laboratorio con 25 clases. Las imágenes originales contienen varios objetos, por lo que el problema se reformula como clasificación de instancias; a partir de las anotaciones se generan recortes individuales, y cada recorte debe clasificarse en una única clase. Así, el sistema completo debe resolver la cadena:

`imagen anotada → crop de objeto → embedding → TN → clase predicha.`

La pregunta principal del trabajo no es si las Tensor Networks sustituyen de manera general a las CNNs o a los Transformers, sino si es posible construir, entrenar y evaluar un clasificador de imágenes basado en TN que funcione de extremo a extremo sobre un problema visual realista, y qué limitaciones aparecen durante ese proceso. A partir de esta pregunta principal se derivan varias cuestiones:

- ¿Qué tipo de representación de imagen permite alimentar una tensor network sin producir secuencias inviables?
- ¿Qué limitaciones aparecen al usar una MPS como punto de partida para imágenes de mayor resolución?

- ¿Puede una estructura jerárquica tipo TTN ofrecer un compromiso razonable entre expresividad y coste computacional?
- ¿Qué aporta combinar contracciones tensoriales con componentes habituales de *deep learning*, como normalización, capas lineales o un *front-end* espacial?
- ¿Qué rendimiento y qué patrones de error se observan al entrenar el modelo sobre un dataset real de clasificación de objetos?

Por tanto, el planteamiento del problema consiste en analizar hasta qué punto las TN son adecuadas para clasificación de imágenes más allá de ejemplos pequeños y controlados.

1.3 Planificación del TFG



Figura 1.1: Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.

- **Semanas 1–3: Introducción y fundamentos de Tensor Networks**

Durante las primeras semanas se adquirirán los conocimientos fundamentales y teóricos sobre Tensor Networks.

- Estudio de la notación y representaciones básicas de Tensor Networks mediante material audiovisual.
- Lectura y análisis de artículos científicos relevantes, con especial atención a modelos tensoriales aplicados a clasificación de imágenes.
- Estudio de apuntes y material teórico relacionado con Tensor Networks y su conexión con la física de sistemas cuánticos.

■ **Semanas 4–6: Estudio de modelos MPS para clasificación de imágenes**

En esta etapa se profundizará en el uso de Matrix Product States como modelos de clasificación de imágenes:

- Análisis del funcionamiento de clasificadores basados en MPS y estudio de sus fuentes de error.
- Revisión de implementaciones existentes y comprensión de los procedimientos de entrenamiento y evaluación.
- Búsqueda y selección de conjuntos de datos adecuados en plataformas como Kaggle o Hugging Face.

■ **Semanas 7–8: Formación en herramientas computacionales**

Durante este periodo se adquirirá el conocimiento práctico necesario para el desarrollo del proyecto:

- Aprendizaje de comandos básicos de Bash y trabajo en entornos de servidor.
- Uso de GitHub para el control de versiones y la organización del código.
- Estudio de buenas prácticas de organización de proyectos y documentación.
- Aprendizaje en profundidad de PyTorch para la implementación de modelos de inteligencia artificial.

■ **Semanas 9–13: Implementación y experimentación**

En esta fase se llevará a cabo la implementación y entrenamiento de los modelos seleccionados:

- Implementación de modelos basados en Tensor Networks para clasificación de imágenes.
- Entrenamiento de los modelos utilizando recursos de computación de altas prestaciones (HPC).
- Evaluación del rendimiento de los modelos y comparación con modelos de inteligencia artificial convencionales.

■ **Semanas 13–15: Análisis de resultados y redacción**

En la etapa final se realizará el análisis de los resultados obtenidos y la documentación del trabajo:

- Análisis crítico de los resultados y discusión de las conclusiones.
- Redacción de la memoria del Trabajo Fin de Grado.
- Organización final del repositorio de código y preparación de la entrega.

1.4 Recursos requeridos

A continuación se enumeran los recursos técnicos utilizados para la ejecución del trabajo:

- Recursos computacionales:
 - Servidor HPC Lorca de la Universidad Europea de Madrid: para el entrenamiento de los modelos con recursos de computación de altas prestaciones.
 - Ordenador personal: para el desarrollo, pruebas locales y escritura de la memoria.
- Herramientas de software
 - Python (lenguaje de programación principal).
 - PyTorch (framework de deep learning).
 - TensorKrowch (librería para implementación de Tensor Networks).
 - NumPy, SciPy, Matplotlib, Seaborn (librerías científicas y de visualización).
 - GitHub (control de versiones y gestión del repositorio de código).
 - Bash / Linux (gestión del entorno HPC y automatización de trabajos).
 - Docker / entornos virtuales (gestión de dependencias y reproducibilidad).
- Recursos bibliográficos y de datos:
 - Artículos científicos de referencia (véase sección de Referencias).
 - Conjuntos de datos públicos: MNIST, OlivettiFaces (obtenidos de repositorios como Kaggle o Hugging Face).

Capítulo 2. OBJETIVOS

2.1 Objetivos generales

El objetivo general del trabajo consiste en estudiar la aplicación de las Tensor Networks como una alternativa dentro del *deep learning* para tareas de clasificación de imágenes, evaluando su viabilidad práctica mediante la implementación, entrenamiento y análisis experimental de modelos tensoriales.

Para ello, el proyecto contempla el estudio del estado del arte en modelos de inteligencia artificial basados en Tensor Networks, el desarrollo de un pipeline de clasificación de imágenes, la implementación de arquitecturas tensoriales entrenables y la evaluación de su comportamiento sobre conjuntos de datos adecuados. El propósito no es demostrar que estos modelos sustituyan de forma general a las arquitecturas convencionales de clasificación de imágenes, sino analizar hasta qué punto pueden utilizarse en un problema realista de clasificación y qué limitaciones aparecen durante su desarrollo.

De forma complementaria, el proyecto persigue adquirir experiencia práctica en el uso de herramientas computacionales propias del ámbito de la investigación en inteligencia artificial, incluyendo el manejo de entornos de entrenamiento, el uso de librerías como PyTorch y TensorKrowch, la ejecución de experimentos en recursos de computación de altas prestaciones y la documentación reproducible del código científico.

2.2 Objetivos específicos

1. Realizar una revisión del estado del arte de los modelos de inteligencia artificial basados en Tensor Networks aplicados a la clasificación de imágenes, analizando las arquitecturas más relevantes y sus principales características.
2. Estudiar los fundamentos teóricos y prácticos de las Tensor Networks, con especial atención a su representación como modelos entrenables y a su integración en tareas de *machine learning*.
3. Diseñar un *pipeline* computacional para clasificación de imágenes, que permita organizar de forma clara la carga de datos, las transformaciones, el modelo, el entrenamiento, la evaluación y el registro de resultados.
4. Implementar modelos o variantes de modelos basados en Tensor Networks para tareas de clasificación de imágenes, utilizando herramientas compatibles con entrenamiento mediante diferenciación automática.
5. Entrenar y evaluar los modelos desarrollados utilizando recursos computacionales

adecuados, familiarizándose con el uso de servidores, entornos de ejecución, dependencias y herramientas básicas de gestión de experimentos.

6. Comparar distintas configuraciones del modelo y del proceso de entrenamiento, analizando cómo afectan decisiones como la representación de la imagen, la dimensión interna del modelo, la regularización o el aumento de datos.
7. Analizar los resultados obtenidos mediante métricas de clasificación, curvas de entrenamiento y diagnósticos que permitan interpretar el comportamiento del aprendizaje y los principales patrones de error.
8. Identificar las principales ventajas y limitaciones de los enfoques tensoriales estudiados en el contexto de la clasificación de imágenes.
9. Documentar el desarrollo del proyecto, describiendo de forma clara los modelos estudiados, las implementaciones realizadas, los experimentos ejecutados y los resultados obtenidos.
10. Preparar y organizar un repositorio de código que recoja las implementaciones y experimentos realizados, favoreciendo su claridad, reproducibilidad y mantenimiento.

2.3 Objetivos que quedan fuera del proyecto

- Estudiar modelos de inteligencia artificial no relacionados con Tensor Networks o no orientados a tareas de clasificación de imágenes.
- Resolver otros problemas del campo de la inteligencia artificial, como detección de objetos, segmentación semántica o generación de imágenes.
- Realizar una comparación exhaustiva frente a todas las arquitecturas convencionales de *deep learning*, o evaluar los modelos en un gran número de benchmarks adicionales.
- Ejecutar los modelos en hardware cuántico real, más allá de enfoques cuántico-inspirados ejecutados en sistemas clásicos.
- Desarrollar aplicaciones orientadas a entornos industriales, sistemas en producción o herramientas de inferencia en tiempo real a partir de los modelos estudiados.

2.4 Beneficios del proyecto

Este proyecto contribuye a ampliar el conocimiento sobre las Tensor Networks como herramienta de aprendizaje automático. Aunque el fundamento teórico de estas arquitecturas está bien establecido en el ámbito de la física cuántica y el álgebra multilineal, su comportamiento práctico en tareas de clasificación de imágenes sigue siendo un área activa de investigación. El trabajo permite analizar qué dificultades aparecen al trasladar estas ideas a un pipeline real de *deep learning*.

Desde el punto de vista computacional, el proyecto permite explorar modelos estructurados en los que la capacidad se controla mediante decisiones como la topología de la red, la dimensión de enlace o la representación de entrada. Esto ofrece una perspectiva complementaria a la de las *deep learning* convencionales, centrada en introducir sesgos estructurales y operaciones tensoriales en el proceso de aprendizaje.

La realización de este proyecto supone también una experiencia práctica en investigación aplicada. Comprende la revisión crítica de literatura científica, el diseño e implementación de experimentos reproducibles, el uso de infraestructuras de entrenamiento y el aprendizaje de herramientas y flujos de trabajo propios del ámbito profesional de la inteligencia artificial. Estos conocimientos son transferibles a entornos académicos y laborales relacionados con la física, la computación científica y el aprendizaje automático.

Capítulo 3. MARCO TEÓRICO

3.1 Deep learning y clasificación de imágenes

Antes de explicar el concepto de *deep learning* es importante entender que es una red neuronal. Una red neuronal es una estructura computacional formada por un conjunto de unidades de procesamiento simples, denominados neuronas. Éstas están interconectadas mediante pesos sinápticos ajustables, llamados en inglés *weights* y *biases*. Cada neurona recibe una serie de entradas, a las cuales llamaremos *inputs*; se calcula una suma ponderada de las mismas y se les aplican una función de activación para producir una salida, la cual llamaremos *output*. El comportamiento de la red, queda determinado por los valores de estos pesos, que son modificados durante el entrenamiento con el objetivo de minimizar una función de error entre los *outputs* y las respuestas esperadas.

Formalizando esta descripción, dada una entrada $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, la operación de una única neurona se expresa como:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \quad (3.1)$$

donde $\mathbf{w} = (w_1, w_2, \dots, w_n)$ es el vector de pesos sinápticos, b es el sesgo (*bias*), y $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación. El argumento de σ es una transformación afín de la entrada, es decir, un producto escalar seguido de una traslación. La no linealidad introducida por σ es esencial, pues sin ella la composición de múltiples capas lineales se reduciría a una única transformación lineal, independientemente de la profundidad de la red [13].

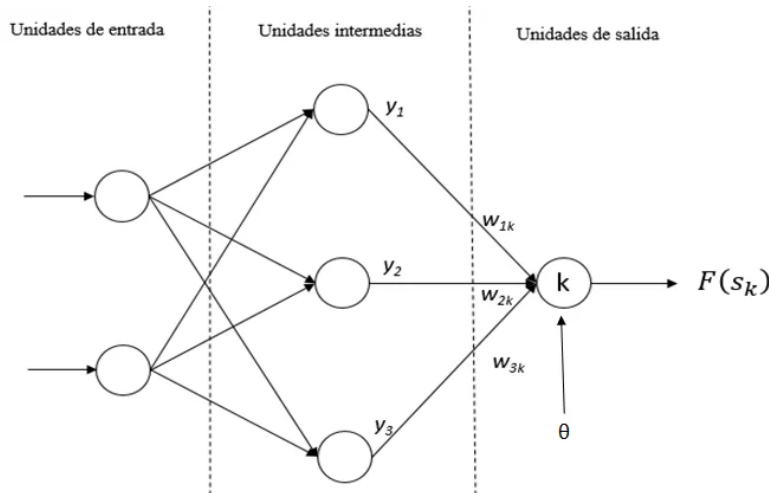


Figura 3.1: Esquema de una red neuronal simple. [1]

Las funciones de activación más utilizadas en la práctica incluyen la sigmoide $\sigma(z) = 1/(1+e^{-z})$, la tangente hiperbólica $\sigma(z) = \tanh(z)$, y la unidad lineal rectificadora (ReLU) $\sigma(z) = \max(0, z)$. La elección de la función de activación tiene consecuencias importantes sobre la dinámica del entrenamiento. Funciones como la sigmoide pueden producir gradientes que se desvanecen en redes profundas, mientras que la ReLU mitiga parcialmente este problema [13].

Una capa completa de la red consiste en m neuronas que operan en paralelo sobre la misma entrada, produciendo un vector de salida $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (3.2)$$

donde $\mathbf{W} \in \mathbb{R}^{m \times n}$ es la matriz de pesos de la capa y $\mathbf{b} \in \mathbb{R}^m$ el vector de sesgos.

Llegamos a la definición de *deep learning*, que no es más que una red neuronal con muchas capas intermedias. La expresión matemática que define una *deep learning* es una extensión de la Ecuación 3.2 pero para L capas:

$$f(\mathbf{x}) = \sigma_L(\mathbf{W}_L \cdots \sigma_2(\mathbf{W}_2 \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_L) \quad (3.3)$$

Los parámetros entrenables del modelo son el conjunto $\{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \dots, \mathbf{W}_L, \mathbf{b}_L\}$, cuyo número total crece con la anchura y la profundidad de la red.

El entrenamiento de una red neuronal se formula como un problema de optimización. Dado un conjunto de entrenamiento $\{(\mathbf{x}_n, y_n)\}_{n=1}^{N_T}$, se define una *función de coste* (o función de pérdida) C que mide la discrepancia entre las predicciones del modelo $f(\mathbf{x}_n)$ y las etiquetas verdaderas y_n . El objetivo es encontrar los parámetros $\boldsymbol{\theta} = \{\mathbf{W}_l, \mathbf{b}_l\}$ que minimizan C :

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{N_T} \sum_{n=1}^{N_T} \mathcal{L}(f(\mathbf{x}_n; \boldsymbol{\theta}), y_n) \quad (3.4)$$

donde $\mathcal{L}$ es una función de pérdida por muestra (por ejemplo, el error cuadrático medio o la entropía cruzada). Este es un problema de minimización en un espacio de alta dimensión, que se resuelve mediante variantes del *descenso de gradiente* [13].

El algoritmo fundamental para calcular los gradientes necesarios es la *backpropagation*, que aplica sistemáticamente la regla de la cadena del cálculo diferencial para propagar el error desde la capa de salida hacia las capas inferiores. Para un parámetro θ cualquiera de la red, el gradiente se calcula como:

$$\frac{\partial C}{\partial \theta} = \frac{\partial C}{\partial \mathbf{y}_L} \cdot \frac{\partial \mathbf{y}_L}{\partial \mathbf{y}_{L-1}} \cdots \frac{\partial \mathbf{y}_{l+1}}{\partial \mathbf{y}_l} \cdot \frac{\partial \mathbf{y}_l}{\partial \theta} \quad (3.5)$$

donde $\mathbf{y}_l$ es la salida de la capa l -ésima. La actualización de los parámetros sigue entonces la regla $\theta \leftarrow \theta - \eta \partial C / \partial \theta$, donde $\eta > 0$ es la tasa de aprendizaje.

La diferencia fundamental entre las redes clásicas y las redes *deep* radica en su capacidad de representación. Una red con una única capa oculta puede, en teoría, aproximar cualquier función continua con la suficiente anchura, pero resulta ineficiente para capturar estructuras jerárquicas, como las que están presentes en las imágenes o en el lenguaje natural [6]. Las *deep learning*, en cambio, aprenden representaciones en múltiples niveles de abstracción. Las capas más bajas detectan características primitivas como bordes o frecuencias, mientras que las superiores combinan estas primitivas para reconocer objetos complejos o conceptos semánticos [5].

En clasificación de imágenes, las redes convolucionales han sido especialmente relevantes porque incorporan localidad espacial y pesos compartidos. Estas dos propiedades son adecuadas para datos visuales, porque los píxeles vecinos tienden a estar relacionados, y los mismos patrones pueden aparecer en distintas regiones de la imagen. La Figura 3.2 muestra algunas topologías básicas de redes neuronales.

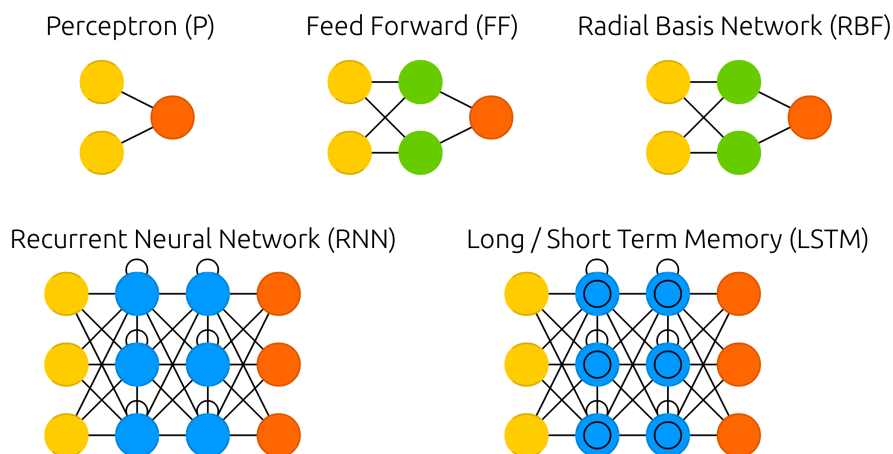


Figura 3.2: Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].

El éxito de estas arquitecturas no elimina sus limitaciones. Muchos modelos modernos alcanzan buenos resultados con un número elevado de parámetros y con un coste computacional considerable. Este TFG se sitúa en ese punto. Estudia si las TN pueden proporcionar modelos de clasificación estructurados, capaces de representar funciones complejas mediante factorizaciones tensoriales controladas.

3.2 Tensores

Antes de profundizar en el mundo de las TN es conveniente sentar primero las bases. Lo primero es entender que un tensor. Un tensor lo podemos ver definido de diferentes formas dependiendo del campo en el que estemos trabajando.

Desde un puesto de vista computacional, los tensores son *arrays* multidimensionales que

generalizan de forma natural los conceptos de escalar, vector y matriz a dimensiones arbitrariamente superiores. Desde un punto de vista físico-matemático, los tensor son objetos matemáticos de orden superior, que representan relaciones multidimensionales entre distintas variables o grados de libertad. Dicho de otra forma, un tensor de orden N es un objeto cuyos elementos tienen N índices y están asociados a N vectores base.

$$R^{I_1 \times I_2 \times \dots \times I_N}$$

donde N es el número de modos y las longitudes $I_1, \dots, I_N$ representan las dimensiones de cada modo. [5]

Bajo esta definición, los escalares son tensores de orden cero, los vectores son tensores de orden 1, y las matrices son tensores de orden 2; todo caso superior constituye propiamente lo que en la literatura se denomina, tensor de alto orden. En la notación convencional, se distinguen los diferentes objetos según su orden: las letras minúsculas en cursiva (a) denotan escalares, las letras minúsculas en negrita ($\mathbf{a}$) denotan vectores, las letras mayúsculas en negrita ($\mathbf{A}$) denotan matrices, y las letras mayúsculas en tipografía cursiva en negrita ($\mathcal{A}$) denotan tensores de orden superior. [5]

La intuición física detrás de los tensores resulta natural cuando se considera, que muchos fenómenos del mundo real son inherentemente multidimensionales. Una imagen en color, por ejemplo, es un tensor de orden tres: altura, anchura y canales cromáticos. Los datos de resonancia magnética funcional (fMRI) constituyen tensores de orden cuatro. En mecánica cuántica, las funciones de onda variacionales de sistemas de muchos cuerpos son, igualmente, tensores de alto orden [5].

El concepto de tensor tiene raíces en el siglo XIX, ligado al desarrollo del cálculo diferencial absoluto por Ricci-Curbastro y Levi-Civita, aunque fue la formulación de la relatividad general por Einstein en 1915 la que consagró los tensores como herramienta fundamental de la física matemática. La descripción matemática de un Tensor es la siguiente:

$$\mathcal{T} = \sum_{i_1, i_2, \dots, i_n} T_{i_1 i_2 \dots i_n} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_n} \quad (3.6)$$

La operación central en el álgebra tensorial es la contracción tensorial. Dados dos tensores $\mathcal{T}$ y $\mathcal{R}$, de orden n y m respectivamente, con elementos $T_{i_1 i_2 \dots i_n}$ y $R_{j_1 j_2 \dots j_m}$ y en la misma base ortonormal $\hat{b}_i$, diremos que el tensor $\mathcal{C}$ es la contracción de ambos tensores por sus último q índices cuando:

$$\mathcal{C} = \sum_{i_1, i_2, \dots, i_{n-q}, j_1, j_2, \dots, j_{m-q}} C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_{n-q}} \hat{b}_{j_1} \hat{b}_{j_2} \dots \hat{b}_{j_{m-q}} \quad (3.7)$$

siendo sus elementos

$$C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} = \sum_{k_1, k_2, \dots, k_q} T_{i_1 i_2 \dots i_{n-q}, k_1 k_2 \dots k_q} R_{j_1 j_2 \dots j_{m-q}, k_1 k_2 \dots k_q} \quad (3.8)$$

De esta definición podemos sacar que la contracción tensorial es la operación mediante la cual dos tensores se contraen en un único tensor a lo largo de sus pares de índices asociados. Existen formas diferentes de contracciones, algunas implican más de dos tensores o un índice repetido en más de dos tensores o incluso repetidos en el mismo tensor. El caso más familiar de contracción tensorial es la multiplicación matricial:

$$\mathbf{AB} = \sum_k A_{ik} B_{kj} \hat{b}_i \hat{b}_j$$

o el producto escalar entre dos vectores:

$$\vec{v} \cdot \vec{w} = \sum_i v_i w_i$$

Entre las operaciones tensoriales fundamentales cabe destacar la traza de una matriz:

$$Tr(A) = \sum_i A_{ii} \quad (3.9)$$

Esto sería la contracción de una matriz consigo misma a través de sus índices repetidos. Debido a esta operación podemos ver que, un tensor también sirve como mapeo entre dos espacios vectoriales. Esto es un tensor, que recibe otro tensor y devuelve un tercer tensor. Por cada índice contraído estamos mezclando componentes del tensor de entrada, reduciendo en 1 el orden del tensor de salida. Por otro lado, los índices que no han sido contraídos aumentarán en 1 el orden del tensor de salida.

3.3 Notación diagramática de Penrose y definición de Tensor Network

La representación algebraica de las contracciones tensoriales, aunque precisa, se vuelve rápidamente inmanejable a medida que aumenta el número de tensores y la complejidad de sus conexiones. Fue Roger Penrose [14] quien, a principios de la década de 1970, introdujo la notación diagramática para tensores, proporcionando una alternativa visual que transformó la manera en que se razona sobre estas estructuras. Los diagramas de redes tensoriales son hoy ampliamente utilizados para describir algoritmos cuánticos y algoritmos de *machine learning* [5].

La idea central de la notación de Penrose es representar cada tensor como un nodo del que emanan tantas aristas como índices tenga el tensor. Concretamente, un nodo con una sola arista representa un vector $\mathbf{a} \in \mathbb{R}^I$; un nodo con dos aristas representa una matriz

$\mathbf{A} \in \mathbb{R}^{I \times J}$; y un nodo con tres aristas representa un tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$. Para indicar que un par de índices tensoriales está siendo contraído, ambas aristas se unen mediante una línea. Por ejemplo, la contracción de un tensor de orden uno con uno de orden dos es la multiplicación matriz-vector habitual. Las aristas que no se conectan a ningún otro tensor se denominan aristas libres, y el número de estas aristas en el diagrama resultante es precisamente el orden del tensor final. Dos nodos que no están unidos mediante aristas, representan el producto tensorial entre ellos.



Figura 3.3: Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia usando la librería Tensor-Network-Editor, creada por Alejandro Mata Ali. [3]

La notación gráfica tensorial ofrece numerosas ventajas sobre la notación tradicional. En forma gráfica, los índices no necesitan nombres ni etiquetas; operaciones como el producto escalar, la traza tensorial y la contracción pueden expresarse sin notación adicional; y el orden del tensor resultante puede leerse simplemente contando el número de aristas libres. En particular, si en el diagrama no quedan aristas libres tras todas las contracciones, el resultado es necesariamente un escalar. Esta propiedad convierte los diagramas en una herramienta de verificación inmediata de la consistencia dimensional de operaciones complejas [7]. Sobre esta base visual puede introducirse ya, de manera natural, el concepto de *Tensor Network (TN)*.

Una TN es, en esencia, una representación gráfica de una ecuación multilineal entre diferentes tensores. Esta representación se apoya en la notación diagramática de Penrose, que con el tiempo se ha ido adaptando a las necesidades de disciplinas como la física, las matemáticas o la ciencia computacional [15]. Puede verse un ejemplo en la Figura 3.4, donde aparece la TN que representa el 2-tensor T_{ip} cuyos elementos se obtienen mediante la operación de contracción

$$T_{ip} = \sum_{j,k,l,m,n,o} A_{ijkl} B_{jlm} C_{kmn} D_{lo} E_{nop} \quad (3.10)$$

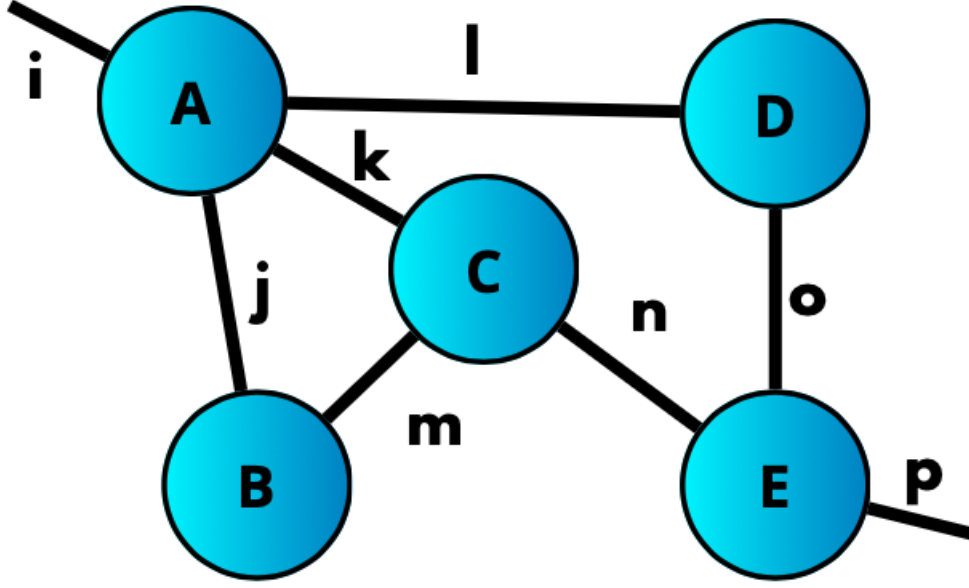


Figura 3.4: Ejemplo de representación diagramática para un tensor. Fuente: [4]

La motivación fundamental para el uso de las Tensor Networks es de naturaleza computacional. Un tensor de orden N con dimensión d en cada nodo tiene d^N componentes, un número que crece exponencialmente con el orden del tensor, haciendo que su almacenamiento y manipulación directos sean computacionalmente prohibitivos; este fenómeno se conoce como la *maldición de la dimensionalidad*. Las TN abordan este problema representando el tensor original no de forma explícita, sino como el resultado de contraer un conjunto de tensores de menor orden, cada uno con un número de parámetros manejable, de modo que la complejidad resultante escala de forma polinomial en lugar de exponencial [5, 7]. Esta idea general se entiende mejor cuando se examina el contexto físico en el que surgió.

3.4 Tensor Networks y reducción de dimensionalidad

Una Tensor Network puede entenderse como una factorización de un tensor de alto orden en una colección de tensores de menor orden conectados entre sí. Su utilidad aparece cuando el tensor completo tendría un número de componentes inmanejable. Si un tensor tiene N modos y cada modo tiene dimensión d , representarlo de forma explícita requiere d^N componentes. Este crecimiento exponencial se conoce como *maldición de la dimensionalidad*.

En física cuántica, el mismo fenómeno aparece al describir sistemas de muchas partículas. Un estado cuántico general de N subsistemas con dimensión local d tiene coeficientes:

$$|\psi\rangle = \sum_{s_1, \dots, s_N} C_{s_1 \dots s_N} |s_1\rangle \otimes \dots \otimes |s_N\rangle. \quad (3.11)$$

El tensor $C_{s_1 \dots s_N}$ tiene d^N componentes. Sin embargo, muchos estados físicamente relevantes poseen una estructura interna que permite representarlos de forma compacta. La idea clave es que no todos los estados ocupan todo el espacio posible con la misma complejidad; sus correlaciones o su entrelazamiento pueden estar limitados [16].

Esta observación se traslada al *machine learning* como una analogía útil. Un modelo de clasificación no necesita representar una función arbitraria sobre todos los píxeles posibles, sino una función que aproveche regularidades de los datos. Las Tensor Networks imponen una estructura de bajo rango o de conectividad concreta sobre el tensor de pesos, reduciendo el número de parámetros si las dimensiones internas, llamadas dimensiones de enlace, permanecen controladas [6, 5].

3.5 Descomposiciones tensoriales fundamentales

Para comprender cómo se construyen las representaciones mediante TN, conviene introducir primero algunas descomposiciones tensoriales básicas. La herramienta algebraica fundamental es la descomposición en valores singulares (SVD). Dada una matriz $\mathbf{A} \in \mathbb{R}^{M \times N}$, la SVD la factoriza como

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \quad (3.12)$$

donde $\mathbf{U}$ y $\mathbf{V}$ son matrices ortogonales, y $\mathbf{\Sigma}$ contiene los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. La importancia de la SVD en TN se debe a que permite obtener la mejor aproximación de rango bajo en norma de Frobenius. Al conservar solo los R valores singulares dominantes se obtiene la matriz de rango R que minimiza el error de reconstrucción, resultado conocido como teorema de Eckart–Young [5].

Una primera generalización del rango matricial a tensores de orden superior es la descomposición CP. Dado un tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$, CP lo aproxima como suma de R tensores de rango 1:

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}, \quad (3.13)$$

donde $\circ$ denota el producto exterior y R es el rango CP. Esta representación es muy compacta, con $R \sum_{n=1}^N I_n$ parámetros, pero presenta dificultades prácticas; determinar el rango CP es un problema NP-difícil y los métodos de ajuste habituales no garantizan alcanzar el óptimo global [17, 5]. En notación diagramática, CP se representa mediante factores vectoriales conectados a un nodo central diagonal.

La descomposición de Tucker es más flexible. En lugar de una única suma de componentes de rango 1, introduce un tensor núcleo $\mathcal{G}$ que modela las interacciones entre los factores:

$$\mathcal{X}_{i_1 i_2 \dots i_N} \approx \sum_{r_1=1}^{R_1} \dots \sum_{r_N=1}^{R_N} \mathcal{G}_{r_1 r_2 \dots r_N} A_{i_1 r_1}^{(1)} A_{i_2 r_2}^{(2)} \dots A_{i_N r_N}^{(N)}. \quad (3.14)$$

Los valores $R_1, \dots, R_N$ son los rangos de Tucker y pueden ser distintos para cada modo. Si las matrices de factores son ortogonales, se obtiene la Higher-Order Singular Value Decomposition (HOSVD), una extensión natural de la SVD al caso tensorial [5]. CP puede verse como un caso particular de Tucker cuando el núcleo es superdiagonal y todos los rangos coinciden. Sin embargo, el núcleo de Tucker contiene $\prod_{n=1}^N R_n$ parámetros, por lo que el coste vuelve a crecer exponencialmente con el orden del tensor.

Esta limitación motiva formatos más escalables como Tensor Train (TT), conocido en física como Matrix Product State (MPS) [18, 16]. TT representa un tensor de orden N como una cadena de tensores locales $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times R_n}$ conectados mediante índices virtuales. Su número de parámetros es

$$\sum_{n=1}^N R_{n-1} I_n R_n,$$

que crece linealmente con N si los rangos se mantienen acotados. Además, la descomposición TT puede construirse mediante SVD sucesivas, lo que proporciona estabilidad numérica y permite controlar el error mediante el truncamiento de valores singulares [5, 18]. Desde la perspectiva cuántica, los rangos TT coinciden con los rangos de Schmidt de las biparticiones correspondientes, conectando esta construcción con la noción de entrelazamiento discutida anteriormente [16].

La Tabla 3.1 resume las diferencias principales entre los formatos anteriores.

Formato	Parámetros	Observaciones
CP	$R \sum_n I_n$	Muy compacta, pero el rango CP es difícil de determinar.
Tucker	$\prod_n R_n + \sum_n I_n R_n$	Flexible, pero el núcleo crece exponencialmente con el orden.
TT/MPS	$\sum_n R_{n-1} I_n R_n$	Escalado lineal para rangos fijos; construible mediante SVD sucesivas.

Tabla 3.1: Comparación entre las descomposiciones CP, Tucker y TT/MPS.

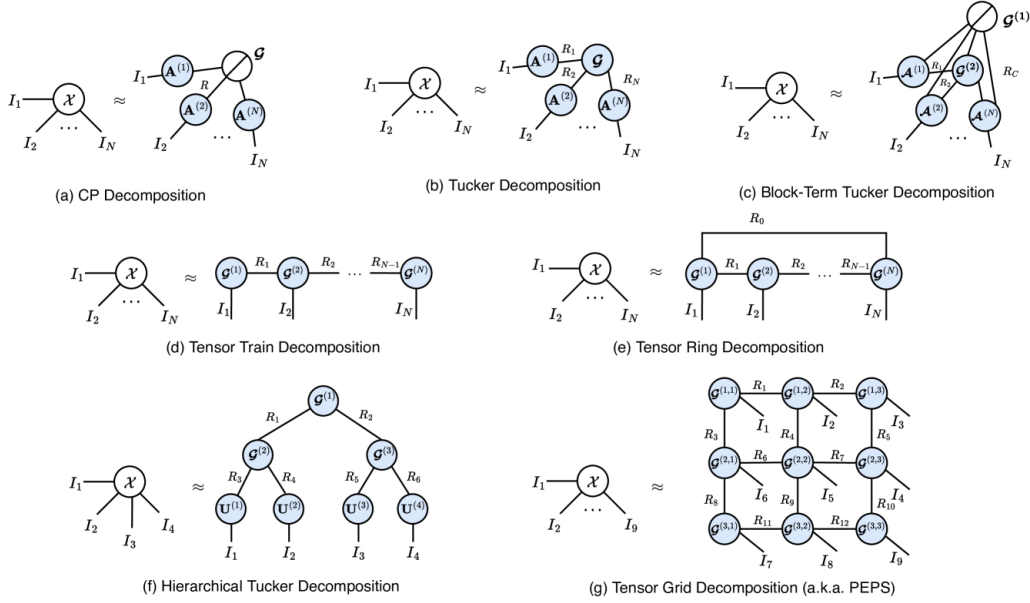


Figura 3.5: Diagramas TN de algunas descomposiciones TN populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (c) La descomposición en términos de bloque descompone un tensor X en una suma de varias descomposiciones de Tucker, con rangos de Tucker bajos. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (e) La descomposición TR descompone un tensor X en un conjunto de tensores núcleo de tercer orden y los contrae en una estructura de anillo. (f) La descomposición HT representa un tensor X como un diagrama con estructura de árbol. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. [5]

3.6 Topologías principales de Tensor Networks

La topología de una Tensor Network determina qué correlaciones puede representar de forma eficiente y con qué coste computacional. En esta sección se detallan las topologías de TN más utilizadas y más relevantes en la literatura.

3.6.1 Matrix Product State / Tensor Train

El *Matrix Product State* (MPS), o Tensor Train (TT), es la topología de TN más estudiada y la que mejor se entiende. Su estructura consiste en una cadena lineal de N tensores de orden 3 conectados secuencialmente a través de índices virtuales de dimensión D (dimension de enlace).

El MPS es la representación más natural de los estados fundamentales de los sistemas cuánticos unidimensionales con interacciones locales, lo que lo hace bastante famoso en el campo de la física.

Las correlaciones que un MPS puede representar decaen exponencialmente con la distancia entre sitios. La función de correlación entre los sitios i y j satisface $\langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle \sim e^{-|i-j|/\xi}$, donde ξ es una longitud de correlación que depende de la dimensión de enlace D [7]. Esto significa que el MPS es especialmente eficiente para capturar correlaciones locales o de corto alcance, pero puede requerir dimensiones de enlace elevadas para representar correlaciones de largo alcance.

El MPS es muy relevante porque combina una interpretación física sólida, algoritmos de contracción manejables e implementaciones maduras. En aprendizaje supervisado, Stoudenmire y Schwab lo utilizaron para parametrizar clasificadores de imágenes sobre MNIST, alcanzando error de test inferior al 1 % en ese conjunto [7]. Su limitación principal es que impone una geometría unidimensional, y una imagen 2D debe linealizarse antes de entrar en la red.

3.6.2 Tree Tensor Network

Una *Tree Tensor Network* (TTN) adopta una estructura jerárquica. Esta estructura, en lugar de propagar la información a lo largo de una cadena, asocia los tensores de la capa más baja (lo que en la literatura se denomina las *hojas* o *leaves* en inglés) a los índices del tensor original. Posteriormente se van contrayendo por pares hacia tensores de capas superiores, reduciendo progresivamente el número de índices hasta llegar a un único tensor raíz, que produce la salida. La distancia entre dos hojas del árbol suele crecer como $\mathcal{O}(\log N)$, lo que puede facilitar la combinación de información lejana respecto a una MPS lineal [5, 10].

Esta propiedad, hace a la TTN especialmente atractiva para este trabajo, porque esta estructura es mucho más eficiente que el MPS para capturar correlaciones de largo alcance. En física, el TTN se utiliza para representar estados con una estructura de entrelazamiento jerárquica, como la que aparece en ciertos sistemas con simetría de escala. En *machine learning*, la estructura de árbol resulta natural para datos con organización jerárquica intrínseca [5].

Sin embargo, el TTN tiene una limitación. Al ser una estructura de árbol sin ciclos, la información entre dos ramas distintas solo puede propagarse a través del nodo ancestro común. Esto limita la capacidad de la red para capturar correlaciones entre nodos que se encuentran en ramas separadas del árbol. Esta restricción motivó el desarrollo de arquitecturas más sofisticadas como la MERA.

3.6.3 MERA

La *Multi-scale Entanglement Renormalization Ansatz* (MERA) fue introducida por Vidal en 2007 como una Tensor Network capaz de representar eficientemente los estados fundamentales de sistemas cuánticos críticos, es decir, sistemas en los que las correlaciones decaen como una ley de potencias en lugar de exponencialmente [19].

La innovación clave de la MERA respecto al TTN es la inclusión de tensores adicionales denominados *disentanglers* (desentrelazadores). En cada capa de la estructura jerárquica, antes de aplicar los tensores de renormalización, que reducen el número de grados de libertad, se aplican *disentanglers* que actúan sobre pares de sitios vecinos para eliminar el entrelazamiento de corto alcance. Los tensores de renormalización se encargan de capturar las correlaciones a escalas mayores. La alternancia de capas de *disentanglers* y capas de renormalización permite a la MERA representar correlaciones a todas las escalas de longitud simultáneamente [5].

La topología MERA destaca por su capacidad para describir de forma eficiente sistemas con correlaciones a múltiples escalas, como ocurre típicamente en sistemas cuánticos complejos. En estos casos, una representación basada en MPS puede requerir una dimensión de enlace que crece de forma polinómica con el tamaño del sistema, mientras que la estructura jerárquica de la MERA permite capturar estas correlaciones de manera más natural [19]. Esta organización por capas está estrechamente relacionada con el concepto de grupo de renormalización, en el que cada nivel de la red puede interpretarse como una descripción efectiva del sistema a una escala de longitud diferente.

En *machine learning*, la topología MERA resulta prometedora para el procesamiento de datos con dependencias a múltiples escalas, como puede ser el caso de las imágenes [5]. Sin embargo, su mayor complejidad estructural (más parámetros y contracciones más costosas que el MPS o el TTN) hace que su uso práctico sea más limitado hasta la fecha.

3.6.4 PEPS

Los *Projected Entangled Pair States* (PEPS) son la generalización más natural del MPS a dos dimensiones. Mientras que en la topología MPS cada tensor se conecta a lo sumo con dos vecinos, en PEPS cada tensor se conecta con cuatro vecinos, formando una red bidimensional que refleja directamente la geometría de una rejilla cuadrada [20].

Para un sistema bidimensional de $M \times N$ sitios, cada tensor local en PEPS tiene un índice físico de dimensión d y cuatro índices virtuales (arriba, abajo, izquierda, derecha) de dimensión D . Esto permite capturar de forma natural las correlaciones espaciales bidimensionales que caracterizan, por ejemplo, los estados fundamentales de sistemas cuánticos definidos sobre rejillas planas [5].

Sin embargo, esta ventaja geométrica tiene un coste. La contracción exacta de una red PEPS es un problema computacionalmente intratable; a diferencia del MPS, donde la contracción exacta tiene un coste polinomial. En la práctica, esto obliga a utilizar métodos

de contracción aproximada, lo que introduce errores adicionales y limita el tamaño de las dimensiones de enlace que pueden utilizarse [5]. Existen aplicaciones de PEPS a clasificación supervisada en conjuntos como MNIST y Fashion-MNIST [8].

Topología	Parámetros típicos	Ventaja principal	Limitación práctica
MPS / TT	$\mathcal{O}(NdD^2)$	Contracción eficiente y literatura consolidada.	Geometría 1D para datos que pueden ser 2D.
TTN	$\mathcal{O}(ND^3)$	Agregación jerárquica y distancia logarítmica.	Interacciones limitadas entre ramas separadas.
PEPS	$\mathcal{O}(NdD^4)$	Geometría 2D natural.	Contracción exacta intratable en general.
MERA	$\mathcal{O}(ND^4)$	Correlaciones multiescala.	Mayor complejidad estructural y computacional.

Tabla 3.2: Comparativa sintética de topologías relevantes de Tensor Networks.

Capítulo 4. METODOLOGÍA

En los capítulos anteriores se han introducido los conceptos matemáticos necesarios para entender el *machine learning*, el *deep learning* y las TN. En esta sección se utilizarán esos conocimientos para explicar como se construye un modelo de *machine learning*. En concreto, un modelo basado en TN.

Este capítulo se centra en la metodología de trabajo, en cómo se organizó el proyecto, como funciona un pipeline de PyTorch, la estructura del código, la preparación del dataset, la forma de definir modelos desde archivos de configuración, el entrenamiento y el registro de cada experimento.

4.1 Pipeline computacional de clasificación en PyTorch

Lo primero que se aprende cuando se empieza a programar con PyTorch es, la estructura de código básica sobre la que se construyen todos los modelos de *machine learning*. En el caso de clasificación de imágenes, la estructura básica es bastante sencilla. Un dataset devuelve una imagen y una etiqueta, una serie de transformaciones preparan la imagen, un *DataLoader* agrupa las muestras en *batches*, el modelo produce *logits*, una función de pérdida compara esos *logits* con la etiqueta correcta y el optimizador actualiza los parámetros mediante *backpropagation*.

```
dataset ->transforms ->DataLoader ->modelo ->loss ->backward ->optimizer
```

Esta estructura es muy importante porque permite separar responsabilidades. El modelo tensorial no necesita saber de dónde viene una imagen, ni el dataset necesita conocer cómo se contrae una TTN. Cada parte tiene una interfaz clara. El dataset entrega tensores de imagen y etiquetas; el modelo recibe un *batch* de imágenes y devuelve una matriz de *logits*; el bucle de entrenamiento se encarga de calcular la pérdida y actualizar los pesos.

En clasificación multiclase se usó **CrossEntropyLoss**. Esta función de pérdida viene por defecto construida en Pytorch, y tiene la ventaja de combinar internamente *LogSoftmax* y la comparación con la etiqueta correcta, por lo que no se necesita aplicar *softmax* al final, en la capa de salida. Esto evita duplicar operaciones y mantiene el entrenamiento numéricamente más estable.

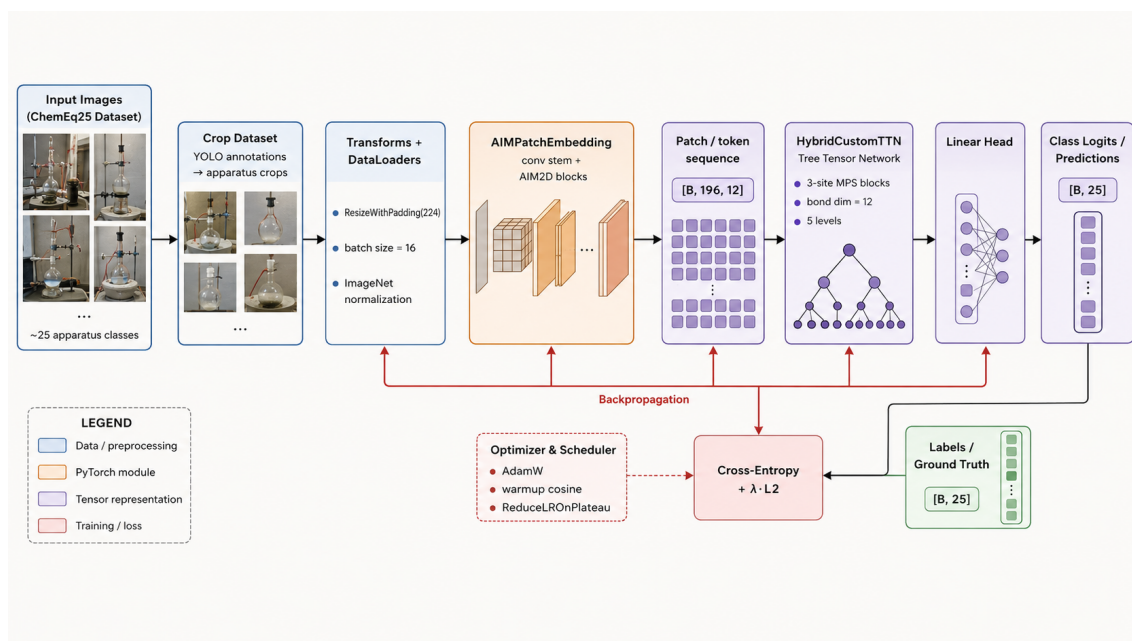


Figura 4.1: Esquema general del pipeline desarrollado en el TFG. A partir de imágenes originales y anotaciones se generan recortes individuales de objetos, se aplican transformaciones y se agrupan en *batches*. Cada imagen procesada entra en el `AIMPatchEmbedding`, que produce una secuencia compatible con la `HybridCustomTTN`. La salida de la red tensorial pasa por una cabeza de clasificación y se compara con la etiqueta real mediante entropía cruzada. Las flechas inferiores representan la actualización de parámetros por *backpropagation*.

4.2 Organización del proyecto

Una parte importante de la metodología es organizar el código de forma que los experimentos fueran modificables sin romper todo el proyecto. En vez de tener un único script grande, el repositorio se estructuró por responsabilidades:

<code>src/tn_dl/</code>	
<code>cli/</code>	puntos de entrada para entrenar, evaluar o auditar
<code>config/</code>	lectura y validación de parámetros desde YAML
<code>data/</code>	datasets, splits, crops, transforms y DataLoaders
<code>embeddings/</code>	transformaciones de imagen a secuencia tensorial
<code>layers/</code>	capas MPS, TTN, HybridCustomTTN y AIM
<code>models/</code>	composición embedding + clasificador tensorial
<code>training/</code>	bucle de entrenamiento, loss, optimizador y scheduler
<code>evaluation/</code>	diagnósticos, matrices de confusión y auditorías
<code>utils/</code>	utilidades generales del proyecto
<code>configs/</code>	ficheros YAML de experimentos
<code>runs/</code>	métricas, curvas, checkpoints y diagnósticos

`scripts/` `scripts` auxiliares y lanzamientos en servidor

Esta separación hace que el proyecto sea mucho más manejable. Por ejemplo, se puede cambiar el *embedding* desde la configuración sin modificar el código del entrenamiento, o se puede comparar una MPS de referencia con una TTN propia manteniendo el mismo flujo de datos. También permite que cada experimento quede definido por un archivo YAML, lo cual es fundamental cuando se prueban muchas combinaciones de hiperparámetros.

Los archivos de configuración YAML incluyen campos como el dataset, el tamaño de imagen, el tipo de *embedding*, `local_dim`, `bond_dim`, el número de épocas, el *learning rate*, la regularización, el *scheduler*, la seed y las opciones de *data augmentation*. De esta forma, un experimento se puede repetir o comparar leyendo su `config.yaml`, sin depender únicamente de notas externas.

4.3 Preparación del dataset ChemEq25

Aunque al principio del proyecto se utilizó Oxford Flowers 102 [21] como banco de pruebas, el dataset principal del TFG es ChemEq25, también llamado en el código *Chemistry Lab Apparatus* [22]. Este dataset contiene imágenes de material de laboratorio y anotaciones asociadas a los objetos presentes en cada imagen.

El problema no se trató como una clasificación directa de imágenes completas debido a que, en muchas imágenes aparece más de un objeto, y esos objetos pueden pertenecer a clases distintas. Si se introdujera la imagen completa en el clasificador, la etiqueta sería ambigua. Por eso se reformuló el problema como clasificación por instancia, en el que cada anotación (YOLO) genera un recorte o *crop* de un único objeto, y ese crop se asocia a una clase.

Tras convertir las anotaciones en muestras individuales, los tamaños de los splits fueron:

Split	Número de crops
Train	4863
Validación	1393
Test	704

El *parser* de anotaciones tuvo que contemplar dos casos. Algunas etiquetas seguían el formato YOLO habitual:

`class_id x_center y_center width height`

pero otras contenían más coordenadas, correspondientes a polígonos de segmentación. Para que todo pudiera tratarse de manera uniforme, las anotaciones de cinco coordenadas se interpretaron como cajas YOLO y las anotaciones con más coordenadas se transformaron en la caja rectangular envolvente del polígono.

Después de obtener el *crop*, todas las imágenes deben convertirse a un tamaño común para entrar al modelo. Inicialmente se podía usar un `Resize(224, 224)` directo, pero esto deforma objetos alargados como pipetas, tubos de ensayo o varillas. Por esa razón se implementó una opción para preservar la relación de aspecto y rellenar hasta obtener una imagen cuadrada. El color de relleno se escogió cercano a la media de *ImageNet*, de forma que tras la normalización el relleno (*padding*) no quedara como un borde artificial demasiado dominante.

4.4 Transformaciones y aumento de datos

El preprocesamiento de las imágenes se separó entre entrenamiento y evaluación. La parte de evaluación consta de un proceso de validación y otro de test. En esta parte no se aplican transformaciones aleatorias, porque se quiere medir el comportamiento del modelo sobre datos estables. En el entrenamiento sí se usa *data augmentation*, con el objetivo de que el modelo no memorice una única apariencia de cada objeto.

Este proceso de aumento de datos se hizo configurable. El usuario puede decidir si quiere aplicarla o no. En ChemEq se usaron opciones como:

- `augmentation_name = none`, sin aumentos aleatorios.
- `augmentation_name = basic`, con transformaciones suaves ya existentes.
- `augmentation_name = light`, añadiendo rotaciones pequeñas y `RandomErasing` bajo.

La configuración final más relevante usó la configuración `light`, rotaciones de 7 grados y `RandomErasing` con probabilidad 0,10. La idea fue introducir variabilidad sin destruir la geometría del objeto, porque en material de laboratorio la forma es precisamente una parte importante de la clase.

4.5 Embeddings

Una TN no recibe una imagen directamente con forma `[C,H,W]`. Antes hay que transformar la imagen en una secuencia de vectores locales:

$$[B, C, H, W] \rightarrow [B, \text{sequence_length}, \text{local_dim}].$$

El primer enfoque fue `PatchEmbedding`. Para imágenes de 224×224 y `patch_size=16`, la imagen se divide en una rejilla 14×14 , es decir, 196 posiciones. Cada *patch* se proyecta con una capa lineal a dimensión `local_dim`. Además, se añadió una ordenación en zigzag para reducir algunas discontinuidades al pasar de una imagen bidimensional a una secuencia unidimensional.

La versión final del proyecto utiliza `AIMPatchEmbedding`. Este embedding procesa primero la imagen como mapa 2D mediante un *stem* convolucional y bloques AIM2D. Después, una convolución 1×1 reduce los canales a `local_dim`, y finalmente la rejilla se aplanan en zigzag:

`[B, 3, 224, 224] -> [B, local_dim, 14, 14] -> [B, 196, local_dim]`.

El bloque AIM2D está inspirado en *Deep Tree Tensor Networks for Image Recognition* [11]. Su idea principal es combinar dos ramas lineales mediante producto elemento a elemento. Si las dos ramas son lineales en la entrada, el producto introduce interacciones bilineales. Esto encaja bien con el lenguaje de redes tensoriales, porque añade interacciones multiplicativas antes de entregar la secuencia a la TTN.

4.6 Arquitectura tensorial

El proyecto mantiene una MPS de referencia y desarrolla una TTN propia. La MPS sirve como punto de comparación técnico, pero la línea principal del trabajo es la versión híbrida de la TTN.

La primera idea de TTN fue construir una red jerárquica de aridad 3. Las posiciones de entrada se agrupan de tres en tres; cada grupo se contrae con un bloque MPS pequeño de tres nodos, y el resultado se usa como entrada del siguiente nivel. El proceso continúa hasta obtener un único vector raíz.

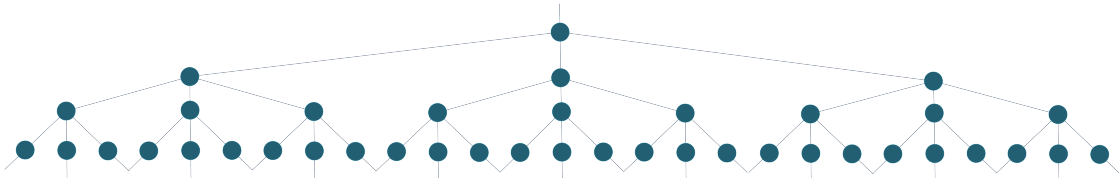


Figura 4.2: Diagrama de la TTN. Es una imagen orientativa, ya que en ella se representan 3 capas cuando la TTN original tiene 5 capas. Esta imagen es de elaboración propia, usando la librería `Tensor-Network-Editor`, creada por Alejandro Mata Ali. [3]

posiciones de entrada -> bloques MPS-3 -> niveles intermedios -> vector raíz

Cada bloque MPS-3 está formado por un nodo izquierdo, uno central y uno derecho. El índice libre de salida se situó en el nodo central, de manera que la salida del bloque tuviera dimensión `bond_dim`. Cuando el número de entradas de un nivel no era múltiplo de 3, se añadieron nodos constantes de padding con tensor $[1, 0, \dots, 0]$. Estos nodos completan los grupos sin introducir parámetros entrenables.

Durante la implementación se comprobó que `TensorKrowch` distingue entre nodos estructurales de la red y nodos de datos creados al introducir un *batch*, por lo que fue necesario

usar `ParamNode` para los nodos entrenables, porque si se crean como nodos ordinarios no aparecen correctamente como parámetros del modelo. Estas validaciones se hicieron con longitudes sintéticas como $L = 9$, $L = 27$ y $L = 196$, comprobando forward, backward y llegada de gradientes.

La versión final divide la TTN en niveles independientes. Cada `TTNLevel` contrae un único nivel de bloques MPS-3 usando `TensorKrowch` y devuelve un tensor PyTorch ordinario. Entre niveles se aplican transformaciones estándar:

```
TTNLevel ->LayerNorm ->Linear ->ReLU ->TTNLevel ->...
```

Por eso se llama `HybridCustomTTN`. Es híbrida porque `TensorKrowch` se usa para las contracciones tensoriales y PyTorch se usa para las transformaciones intermedias, el entrenamiento y la integración general del modelo. La salida final de la TTN es un vector raíz de dimensión `bond_dim`, que pasa por una cabeza de clasificación lineal para obtener los *logits*.

4.7 Entrenamiento y regularización

El entrenamiento se realizó con `AdamW` y `CrossEntropyLoss`. Además del *weight decay* propio de `AdamW`, se implementó una regularización L_2 explícita configurable. Esta regularización se suma a la pérdida durante entrenamiento, pero las métricas registradas como `train_loss` conservan la entropía cruzada pura para que sean comparables.

También se añadió *label smoothing*. En entrenamiento se usa `CrossEntropyLoss` con suavizado de etiquetas, mientras que en evaluación se mantiene la entropía cruzada normal. Así, el modelo se entrena con etiquetas ligeramente suavizadas, pero la `eval_loss` sigue midiendo el ajuste real frente a etiquetas duras.

El *learning rate* se controló con dos mecanismos. Primero, un *warmup* configurable, lineal o coseno, que evita empezar el entrenamiento con pasos demasiado grandes. Después, `ReduceLROnPlateau`, que reduce el *learning rate* cuando la `eval_loss` deja de mejorar durante varias épocas. El estado del optimizador y del *scheduler* se guarda en el checkpoint para poder reanudar correctamente un entrenamiento.

4.8 Métricas, diagnósticos y trazabilidad

Cada entrenamiento genera una carpeta dentro de `runs/`. En ella se guardan la configuración resuelta, el checkpoint del mejor modelo, el archivo `metrics.csv`, las curvas de pérdida y los diagnósticos. Esto fue fundamental para poder reconstruir qué se había probado y con qué parámetros.

Las métricas principales registradas fueron:

- `train_loss` y `eval_loss`;

- `train_accuracy` y `eval_accuracy`;
- `eval_top5_accuracy`;
- confianza media en aciertos y errores;
- precisión, *recall* y F1 macro en las versiones finales.

Además, se generaron diagnósticos por clase: matriz de confusión, métricas por clase, distribución de predicciones, principales confusiones, curvas precision-recall y curvas F1-confidence. En este TFG estas métricas se interpretan como métricas de clasificación de crops. Esto es importante porque el paper de ChemEq25 evalúa modelos de detección, mientras que aquí el modelo clasifica recortes ya generados a partir de las anotaciones.

4.9 Ejecución en servidor

Todos los entrenamientos se ejecutaron en los servidores Lorca de la universidad. Esta parte también forma parte de la metodología del proyecto, porque permitió lanzar experimentos largos, usar GPU, guardar checkpoints y analizar las métricas posteriormente.

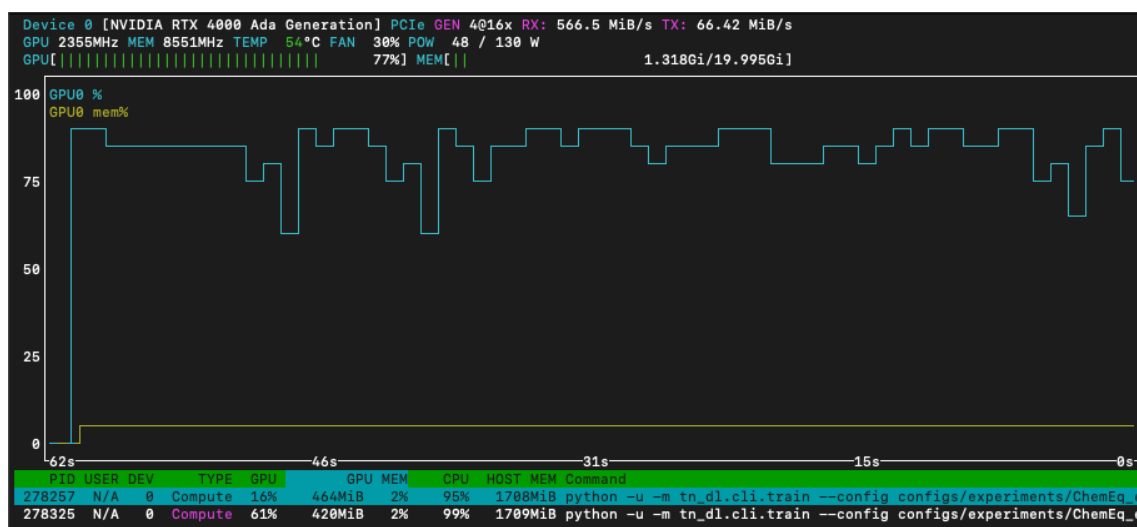


Figura 4.3: Captura de una GPU NVIDIA RTX 4000 Ada Generation del servidor Lorca de la UEM ejecutando dos procesos de entrenamiento en paralelo.

El trabajo en el servidor obligó a cuidar el entorno de Python, las dependencias, los scripts de lanzamiento y los logs. También hizo más importante que cada run guardara su propia configuración, ya que muchas pruebas se lanzaron en paralelo o en días distintos. Sin esa trazabilidad, habría sido muy difícil distinguir si una mejora venía de la arquitectura, del preprocesamiento, de la seed, del *learning rate* o de la regularización.

Capítulo 5. RESULTADOS

En este capítulo se exponen los resultados que se han ido obteniendo a lo largo del proyecto. La idea es ir reconstruyendo el camino experimental que llevó a obtener los resultados finales. Se explicará qué ideas se probaron primero, qué problemas surgieron, qué hipótesis aparecieron a partir de los datos y qué cambios fueron mejorando el modelo.

Hay una aclaración importante antes de empezar. El paper de ChemEq25 compara modelos de detección como *YOLOv11* o *RF-DETR* [22]. En este TFG el planteamiento final no es detección, sino clasificación de crops generados a partir de las anotaciones YOLO. Por tanto, las métricas como *accuracy*, *F1* o *classification_mAP* sirven para evaluar nuestro clasificador, pero no son directamente equivalentes al *mAP@50* de un detector. Aun así, sí permiten demostrar que un modelo de *deep learning* basado en TN puede funcionar de forma competitiva en un problema de clasificación de imágenes.

5.1 Primeros experimentos con Flowers102

Flowers102 fue el dataset de prueba inicial. No era el objetivo final del TFG, pero sirvió para comprobar que el pipeline entrenaba, que las métricas se guardaban bien y que la red tensorial recibía gradientes. Además, al tener 102 clases, era un buen primer dataset de prueba para detectar si el modelo estaba aprendiendo algo o si se quedaba cerca del azar.

5.1.1 Modelo original CustomTTN

Con el modelo original **CustomTTN** se hicieron los primeros experimentos. Los resultados fueron los siguientes:

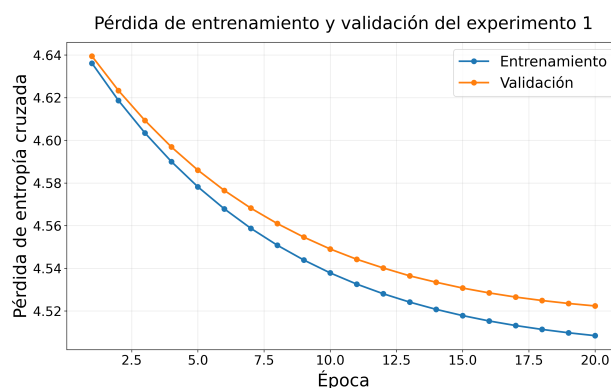


Figura 5.1: Curvas de pérdida del primer experimento con **CustomTTN** sobre Flowers102.

Experimento	Mejor acc. eval	Época	Mejor eval loss	Eval loss final
1	3,58 %	9	4,5153	4,5156

Tabla 5.1: Resumen representativo del primer experimento con CustomTTN.

En este momento se estaba usando Flowers102. Si calculamos $\ln(102)$ se obtiene aproximadamente 4,625, que está muy cerca de la pérdida que aparece en la gráfica. Esto significa que el modelo estaba muy próximo a un comportamiento aleatorio. La *train loss* bajaba muy poco y la precisión de evaluación se quedaba alrededor del 3,6 %. Por tanto, aunque el código ya funcionaba, el modelo todavía no estaba aprendiendo una representación útil.

5.1.2 Hipótesis 1

A raíz de estos resultados se elaboró la primera hipótesis. El modelo no aprendía bien porque entre las contracciones tensoriales no había funciones no lineales como ReLU, sigmoide o SeLU. Esta hipótesis tenía sentido porque, como se vio en el marco teórico, las redes neuronales profundas no se basan solo en aplicar transformaciones lineales, sino en intercalar activaciones no lineales que aumentan su capacidad expresiva.

La idea inicial fue aplicar estas funciones en cada salida resultante de la contracción de cada bloque MPS. Sin embargo, esto no era tan directo en TensorKrowch, porque los nodos resultantes de una contracción no se pueden modificar como si fueran tensores PyTorch normales. Por eso fue necesario cambiar la estructura del modelo y dividir la TTN en niveles independientes.

5.1.3 Modelo HybridCustomTTN

Después de aplicar el cambio de la hipótesis 1, se lanzó un nuevo experimento. El modelo ya no era una TTN monolítica, sino una arquitectura híbrida; cada nivel contraía bloques MPS-3 con TensorKrowch y entre niveles se aplicaban transformaciones PyTorch del tipo `LayerNorm ->Linear ->ReLU`.

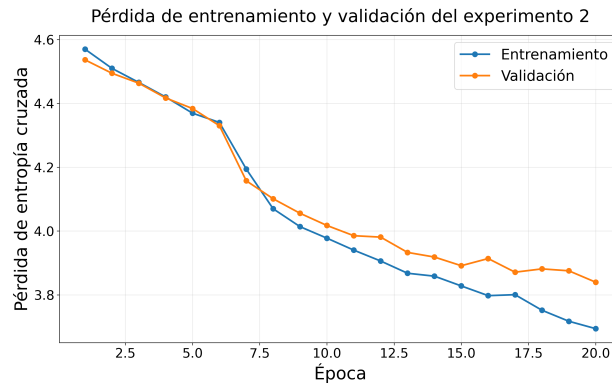


Figura 5.2: Primer experimento con la arquitectura híbrida. La pérdida ya no queda tan cerca del comportamiento aleatorio inicial.

Experimento	Mejor acc. eval	Época	Mejor eval loss	Eval loss final
2	13,60 %	47	3,5593	5,1254

Tabla 5.2: Resumen del primer experimento relevante con HybridCustomTTN.

Este resultado confirmó la hipótesis 1, ya que el modelo dejó de tener un comportamiento prácticamente aleatorio. La precisión subió de 3,58 % a 13,60 %, y la mejor *eval loss* bajó de 4,5153 a 3,5593. La mejora era clara, pero también apareció una señal nueva; al avanzar el entrenamiento, la pérdida de evaluación volvía a empeorar.

Para entender mejor esta tendencia se repitió el experimento durante más épocas:

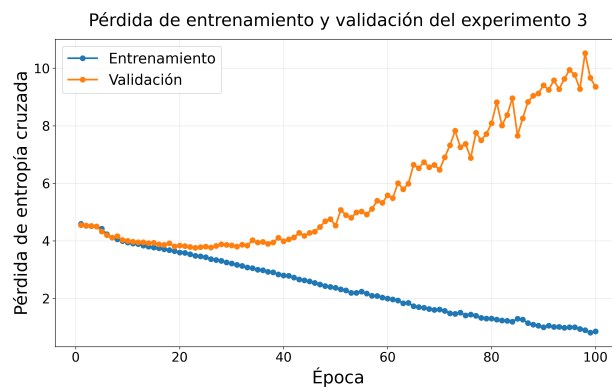


Figura 5.3: Experimento largo con HybridCustomTTN sobre Flowers102.

Experimento	Acc. eval	Época	Train loss	Eval loss	Acc. final
3	12,46 %	54	1,3295	8,2368	8,55 %

Tabla 5.3: Resumen del experimento largo con HybridCustomTTN.

Aquí ya se ve mucho mejor el problema. La *train loss* baja de forma sostenida hasta 1,3295, así que el modelo sí aprende el conjunto de entrenamiento. Sin embargo, la *eval loss* sube hasta 8,2368 y la precisión final cae al 8,55 %. Esto es una señal clara de *overfitting*, el modelo se adapta al entrenamiento, pero no generaliza bien a imágenes nuevas.

5.1.4 Hipótesis 2

La segunda hipótesis fue que el modelo estaba perdiendo demasiada información espacial de la imagen. Hasta ese momento se usaba `PatchEmbedding`. Para imágenes de 224×224 y `patch_size=16`, la imagen se divide en una rejilla de 14×14 , es decir, 196 posiciones. Cada *patch* se proyecta a una dimensión `local_dim` y se entrega a la TTN:

$$[B, 3, 224, 224] \rightarrow [B, 196, \text{local_dim}].$$

El problema es que, en las primeras pruebas, `local_dim` era muy pequeña. Se estaba comprimiendo cada *patch* de la imagen a muy pocos números antes de que el modelo pudiera procesar bien la localidad espacial. El zigzag ayudaba a ordenar la secuencia, pero no recuperaba la estructura 2D original. Por eso el siguiente cambio fue introducir un *embedding* que procesara la imagen como mapa bidimensional antes de aplanarla.

5.2 Cambio del embedding a AIM

El cambio a `AIMPatchEmbedding` nace de la hipótesis 2. El objetivo era conservar más información espacial antes de entregar la imagen a la TTN. Para ello se incorporó un *front-end* inspirado en los bloques AIM de *Deep Tree Tensor Networks for Image Recognition* [11].

$$\begin{aligned} \text{imagen} &\rightarrow \text{stem CNN} \rightarrow \text{AIM2D} \rightarrow \text{Conv2d } 1 \times 1 \rightarrow \text{flatten} + \text{zigzag} \\ &\rightarrow \text{HybridCustomTTN} \end{aligned}$$

El bloque AIM2D combina dos ramas lineales mediante producto elemento a elemento. Si ambas ramas son lineales en la entrada, su producto introduce interacciones bilineales. Esta idea encaja muy bien con las redes tensoriales, porque introduce interacciones multiplicativas estructuradas antes de pasar a la TTN.

En términos de dimensiones, con imágenes de 224×224 y `patch_size=16`, el front-end reduce la imagen a una rejilla 14×14 . Después, una convolución 1×1 proyecta los canales a `local_dim`, y finalmente la rejilla se aplanar:

$$[B, 3, 224, 224] \rightarrow [B, \text{local_dim}, 14, 14] \rightarrow [B, 196, \text{local_dim}].$$

El primer experimento con AIM durante 50 épocas fue:

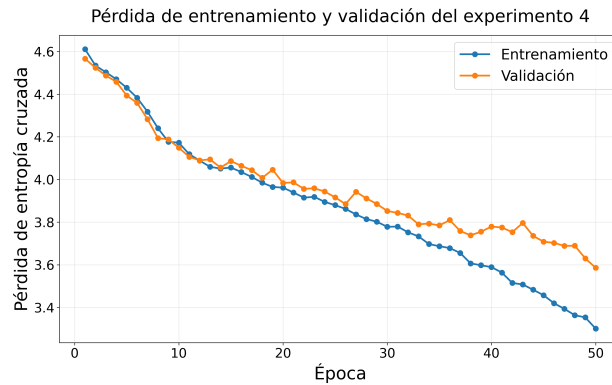


Figura 5.4: Experimento con AIMPatchEmbedding durante 50 épocas.

Experimento	Épocas	Mejor acc. eval	Mejor eval loss	Acc. final
4	50	14,01 %	3,5859	14,01 %

Tabla 5.4: Primer resultado con AIMPatchEmbedding.

El resultado fue positivo porque la pérdida seguía bajando y no se observaba una separación tan violenta entre entrenamiento y evaluación. Para compararlo mejor con los experimentos anteriores, se repitió con 100 épocas:

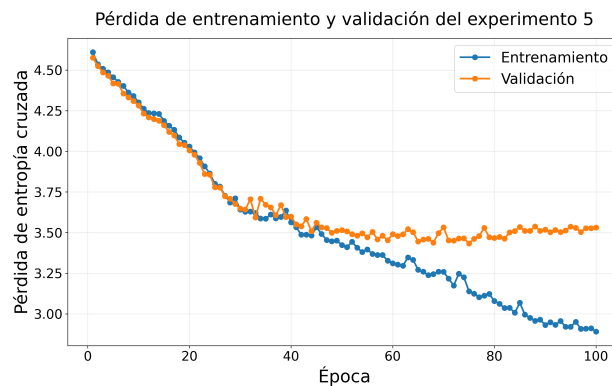


Figura 5.5: Experimento con AIMPatchEmbedding durante 100 épocas.

Experimento	Épocas	Mejor acc. eval	Mejor eval loss	Eval loss final
5	100	15,88 %	3,4330	3,5312

Tabla 5.5: Resultado con AIM durante 100 épocas.

Comparado con [Figura 5.3](#), el *overfitting* ya no era tan agresivo. Antes la *eval loss* acababa en 8,2368, mientras que con AIM se mantuvo alrededor de 3,5312. La precisión seguía

siendo baja, pero el comportamiento del entrenamiento era más sano. Esto indicaba que AIM no era una solución mágica, pero sí estaba atacando el problema correcto.

El siguiente paso fue aplicar *data augmentation*. Esta técnica consiste en generar variaciones de las imágenes durante el entrenamiento, por ejemplo con cambios de recorte, color, orientación o borrado parcial. La idea es que el modelo no memorice una única versión de cada imagen, sino que aprenda rasgos más generales.

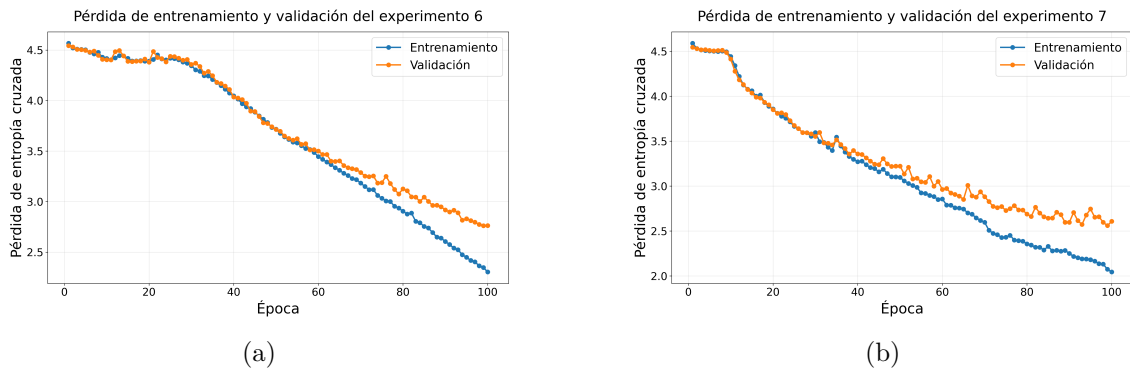


Figura 5.6: Experimentos en Flowers102 con AIM y *data augmentation*.

Experimento	Mejor acc. eval	Top-5 eval	Mejor eval loss	Acc. final
6	30,13 %	61,40 %	2,7612	29,97 %
7	33,96 %	66,04 %	2,5600	33,96 %

Tabla 5.6: Mejores resultados sobre Flowers102 después de añadir AIM y *data augmentation*.

Aquí el cambio ya fue bastante más claro. La precisión subió hasta 33,96 % y el top-5 alcanzó 66,04 %. Aun así, la matriz de confusión mostraba que el modelo seguía confundiendo muchas clases.

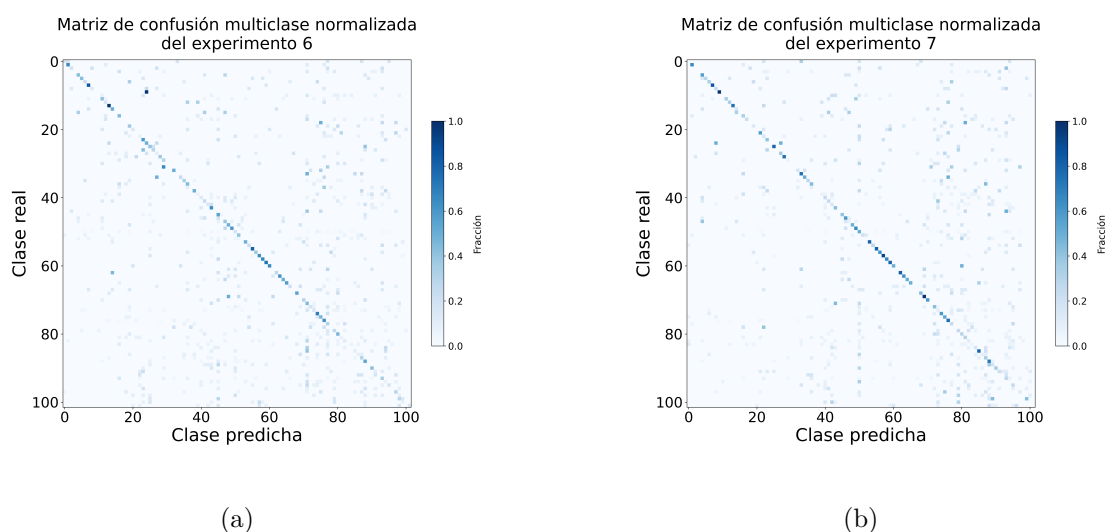


Figura 5.7: Matrices de confusión multiclase en Flowers102 tras AIM y *data augmentation*.

En una matriz de confusión ideal se vería casi todo concentrado en la diagonal. En este caso la diagonal ya aparece, lo cual significa que el modelo no está prediciendo completamente al azar, pero sigue habiendo muchos valores fuera de ella. Esto llevó a la tercera hipótesis.

5.2.1 Hipótesis 3

La tercera hipótesis fue que el problema ya no era solo que el modelo no aprendiera, sino que Flowers102 era un dataset difícil para esta fase del proyecto. Tiene 102 clases de flores, muchas de ellas visualmente muy parecidas. Incluso para una persona puede ser complicado distinguir correctamente todas las clases sin un conocimiento previo sobre botánica.

Además, Flowers102 nunca fue el objetivo central del TFG. El objetivo era trabajar con imágenes de material de laboratorio, usando ChemEq25. Por tanto, una vez comprobado que el pipeline funcionaba y que AIM mejoraba el comportamiento, el siguiente paso natural fue cambiar al dataset principal.

5.3 Cambio de dataset a ChemEq25

Antes de implementar cualquier dataset en un modelo, es muy importante revisarlo detenidamente. La parte de limpiar y adaptar los datos es una de las más importantes dentro del *machine learning*, porque si los datos no están bien preparados, es muy difícil obtener buenos resultados después.

Revisando ChemEq25 apareció una particularidad importante. Muchas imágenes tenían dimensiones de 640×640 píxeles, pero podían contener más de un objeto de laboratorio.

Incluso había fotografías con muchos objetos distintos en la misma escena. Si se introdujera una imagen completa en el clasificador, el modelo no sabría qué objeto debe clasificar. Por eso fue necesario usar las anotaciones YOLO del dataset.

Las anotaciones YOLO indican dónde está cada objeto dentro de la imagen. Con ellas se generaron recortes que contienen un único objeto, y cada recorte mantiene su etiqueta correspondiente.

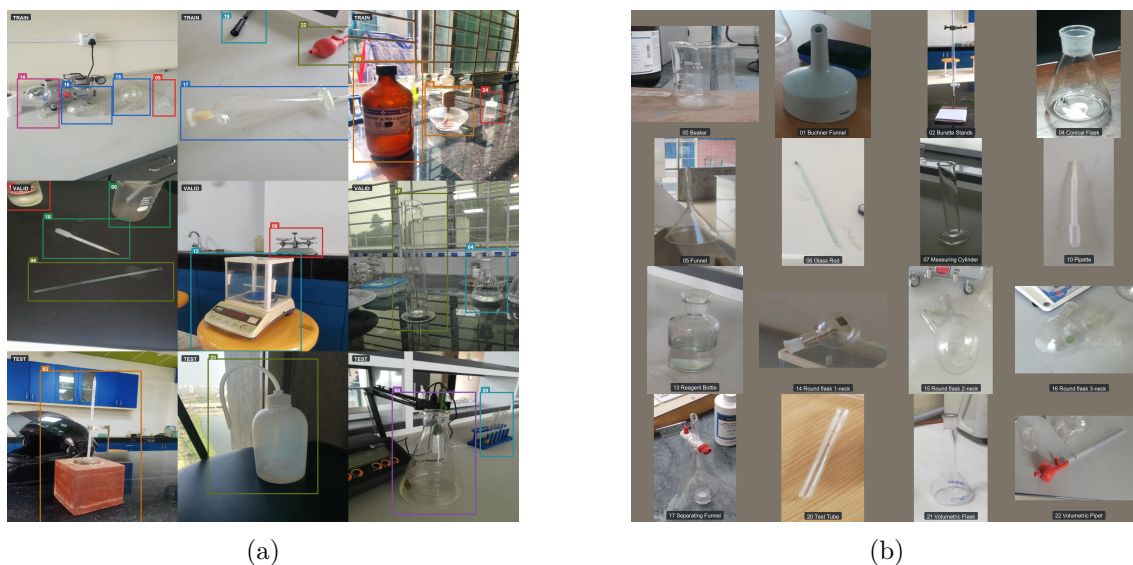


Figura 5.8: En la figura a) vemos ejemplos de cajas obtenidas a partir de las anotaciones YOLO. En la figura b) vemos ejemplos de los recortes utilizados como entrada del clasificador. El color gris que se ven entre las imágenes es el color de relleno cercano a la media de *ImageNet*. Las imágenes en b) son los recortes tal y como las va a ver el modelo.

Una vez preparado el dataset, se lanzó el primer experimento serio con ChemEq25:

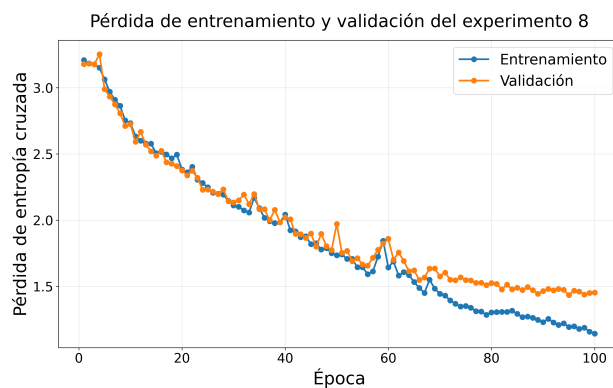


Figura 5.9: Primer experimento con ChemEq25.

Experimento	LR	Acc. eval	Top-5	Eval loss	Acc. final
8	0,0005	57,00 %	86,22 %	1,4343	56,86 %

Tabla 5.7: Primer resultado sobre ChemEq25.

El salto respecto a Flowers102 fue muy grande. La precisión pasó de 33,96 % en Flowers a 57,00 % en ChemEq25, y el top-5 alcanzó 86,22 %. Esto confirmó la hipótesis 3. El problema de Flowers102 no era solo la arquitectura, sino también la dificultad de separar 102 clases de flores muy parecidas. Con 25 clases de material de laboratorio, el modelo empezó a mostrar mucha más capacidad.

5.3.1 Efecto del learning rate

El siguiente cambio fue reducir el *learning rate* de 0,0005 a 0,00025. El *learning rate* es el tamaño del paso que da el optimizador al actualizar los parámetros. Si es demasiado alto, el modelo puede moverse de forma brusca por el espacio de parámetros y no estabilizarse bien. Si es demasiado bajo, el entrenamiento puede avanzar muy lento.

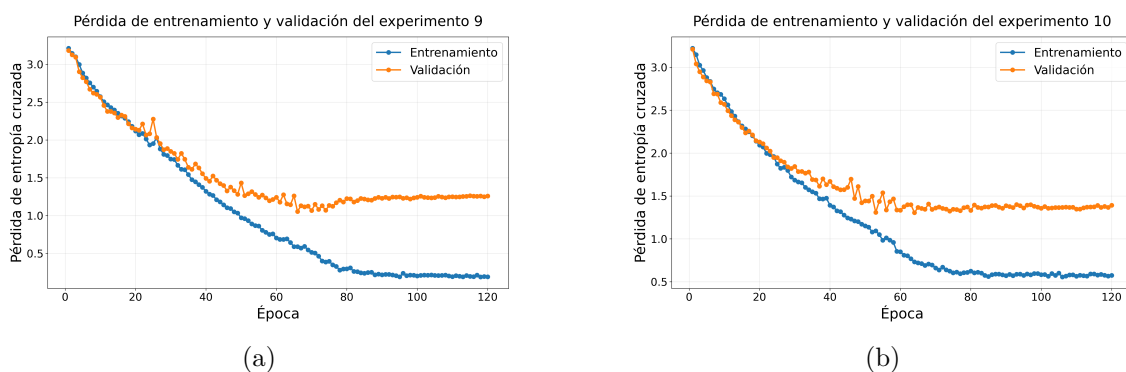


Figura 5.10: Mismo experimento que [Figura 5.9](#), pero con *learning rate* menor.

Experimento	Cambio	Acc.	Top-5	Loss	Train final	Eval final
8	LR 0,0005	57,00 %	86,22 %	1,4343	64,80 %	56,86 %
9	LR 0,00025	75,23 %	93,90 %	1,0539	94,78 %	74,66 %
10	MLP head	70,57 %	94,33 %	1,3047	80,34 %	69,56 %

Tabla 5.8: Comparación de los primeros ajustes sobre ChemEq25.

La bajada del *learning rate* produjo una mejora muy fuerte: de 57,00 % a 75,23 %. Esto no significa que el modelo empezara desde un punto distinto, porque la *seed* era la misma. Lo que cambió fue la trayectoria de optimización. Con pasos más pequeños, el modelo encontró una zona mucho mejor.

También se probó una cabeza MLP en vez de una cabeza lineal. La motivación era que el

top-5 era alto, así que quizá la representación final contenía información suficiente, pero faltaba una separación más fina entre clases. Sin embargo, la MLP no mejoró el resultado, se quedó en 70,57 %. Por eso se mantuvo la cabeza lineal como opción principal.

En el experimento con *learning rate* 0,00025 apareció otro patrón interesante. La mejor *eval loss* se alcanzó en la época 66, pero la mejor precisión llegó en la época 104. Esto indica que el modelo acertaba más ejemplos, pero cuando fallaba podía hacerlo con más confianza. Esa lectura motivó los siguientes cambios, *warmup*, regularización y *label smoothing*.

5.4 Ajustes de entrenamiento y regularización

A partir de este punto hubo muchos experimentos. Por cada experimento que se hacía, se iban atacando los problemas observados. Optimización inestable, sobreconfianza, clases parecidas y sensibilidad a la seed.

Experimento	Cambio principal	Mejor acc.	Top-5	Mejor loss	Acc. final
11	Warmup con configuración 8/8	78,46 %	94,62 %	1,0915	78,03 %
12	Subida a dimensión 12/12	81,55 %	95,41 %	0,8637	80,33 %
13	Warmup coseno, preservación de aspecto y L_2 suave	86,86 %	97,77 %	0,7065	86,43 %
14	Label smoothing 0,05, seed 42	87,08 %	96,77 %	0,5221	86,93 %
15	Repetición del experimento con label smoothing 0,05	88,94 %	97,20 %	0,4617	88,44 %

Tabla 5.9: Evolución de los ajustes de entrenamiento antes del último cambio de *data augmentation*.

El *warmup* ayudó a que el entrenamiento no empezara de forma tan agresiva. Subir *local_dim* y *bond_dim* a 12 también mejoró bastante, porque la representación que entraba en la TTN y el vector interno de enlace tenían más capacidad.

Después, la combinación de warmup coseno mas la preservación de relación de aspecto de las imágenes recortadas y la implementación de un término de regularización L_2 , llevó el modelo hasta 86,86 % de precisión. En este punto el modelo ya funcionaba bien, pero seguía habiendo sobreconfianza en los errores. Por eso se incorporó *label smoothing*. Con *label_smoothing*=0.05 no solo mejoró ligeramente la precisión, sino que bajó de forma clara la pérdida de evaluación y la confianza media cuando el modelo se equivocaba.

Las clases difíciles seguían siendo bastante coherentes entre experimentos, objetos alargados o muy parecidos entre sí, como *Measuring Cylinder*, *Volumetric Flask*, *Separating*

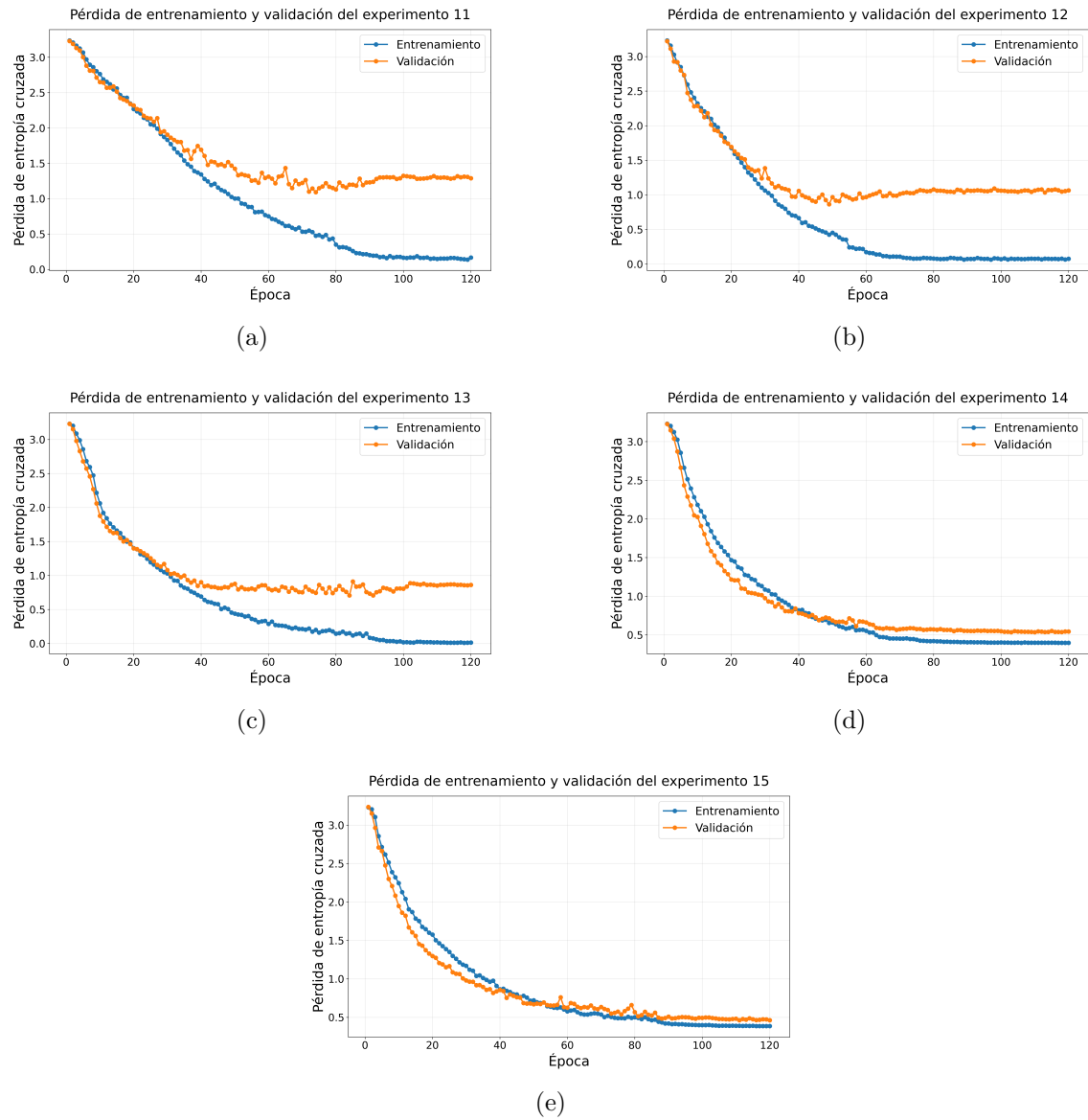


Figura 5.11: Distintos experimentos a los que se le fueron aplicando pequeñas mejoras detalladas en [Tabla 5.9](#). Se puede ver como las curvas de perdida van mejorando con cada experimento.

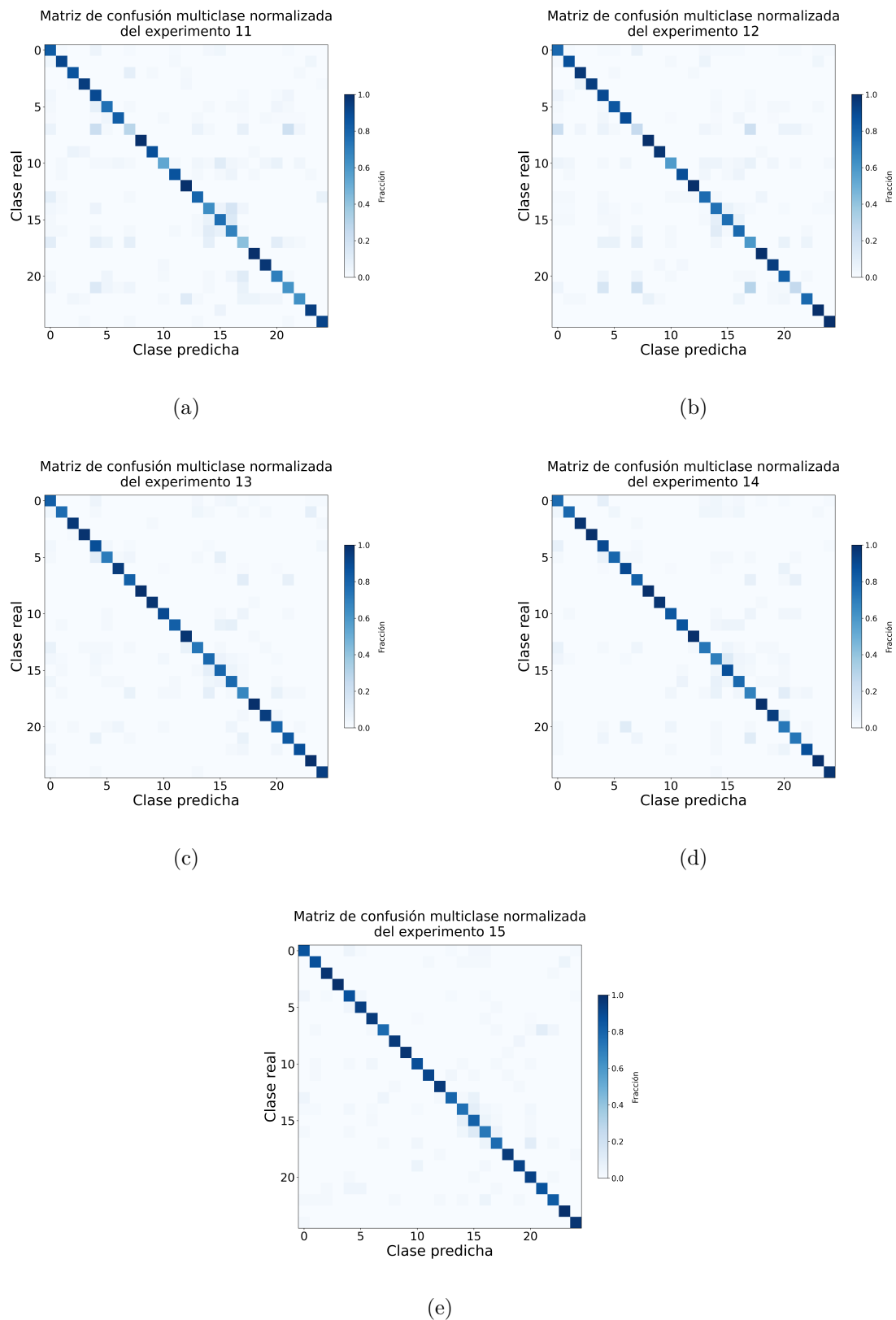


Figura 5.12: Matrices de confusión de distintos experimentos a los que se le fueron aplicando pequeñas mejoras detalladas en [Tabla 5.9](#). Se puede ver como la diagonal se hace más evidente a medida que se aplican los cambios.

Funnel, *Test Tube*, *Pipette* y las distintas variantes de *Round Bottom Flask*. Esto era una señal importante, los errores no eran aleatorios, sino que estaban concentrados en formas visualmente parecidas.

5.4.1 Hipótesis 4

La cuarta hipótesis fue que el modelo no necesitaba solo más capacidad, sino ver variaciones suaves de esos objetos durante el entrenamiento. Si el modelo confundía clases parecidas por pequeñas diferencias de forma, orientación o visibilidad, entonces una *data augmentation* más adecuada podía ayudar a aprender representaciones más robustas.

Por eso se introdujo una opción de aumento **light**, con rotaciones pequeñas y borrado aleatorio bajo. No se buscaba deformar agresivamente los objetos, porque en este dataset la geometría importa mucho. La idea era justamente la contraria, simular variaciones más ligeras sin destruir la clase.

5.5 Salto con data augmentation light

El cambio más importante de toda la fase final fue la nueva *data augmentation* suave. Manteniendo la configuración fuerte anterior, se lanzaron cuatro experimentos con seeds distintas:

Experimento	Seed	Mejor acc.	Top-5	Mejor loss	Macro F1
16	55	94,54 %	98,85 %	0,2684	94,64 %
17	42	94,19 %	99,21 %	0,2503	94,26 %
18	69	96,05 %	99,43 %	0,1997	95,94 %
19	111	94,26 %	98,71 %	0,2634	94,28 %
Media	–	94,76 %	–	–	94,78 %

Tabla 5.10: Resultados con `augmentation_name = light`.

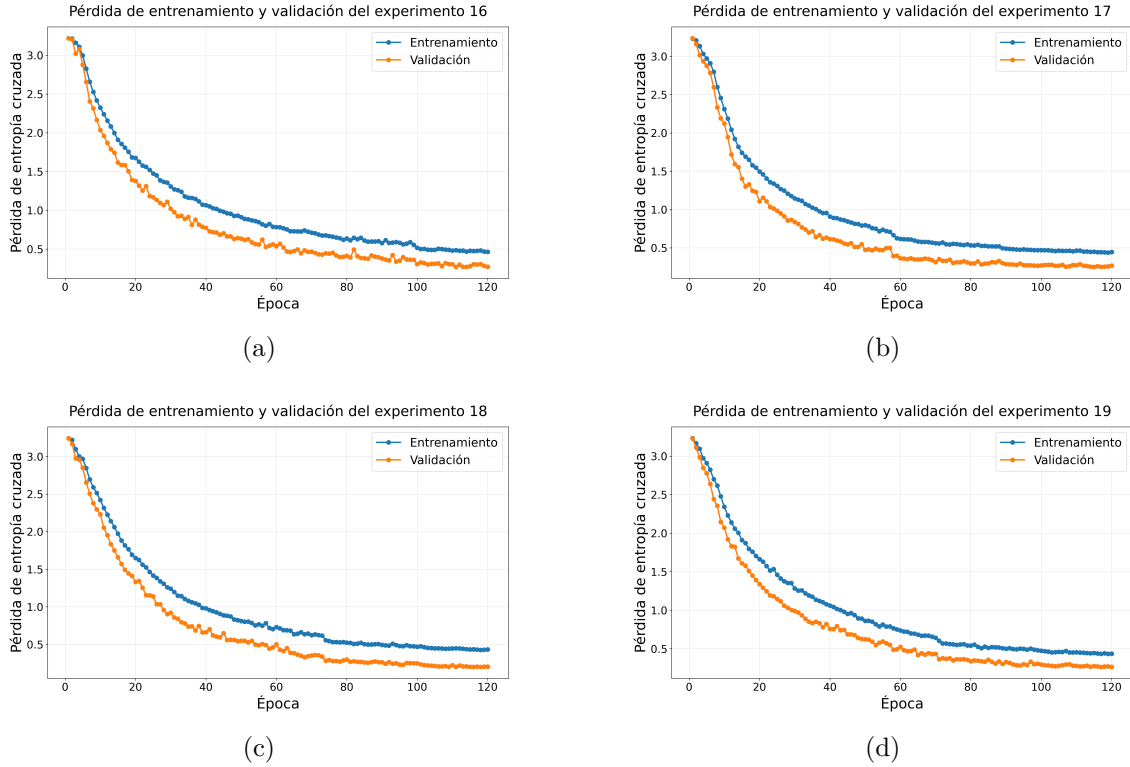


Figura 5.13: Experimentos realizados con el cambio del *data augmentation*

Este fue el salto más grande del proyecto. Antes de esta modificación, el mejor resultado estaba alrededor de 88,94 %. Con *data augmentation light*, los cuatro experimentos superaron el 94 %, y la mejor llegó a 96,05 %. Estos resultados confirman las hipótesis 4; la preparación de datos y la regularización visual fueron decisivas.

5.6 Mejor modelo obtenido

El experimento con mejores resultados es el número 18. La configuración principal fue la siguiente. *Seed* 69, dimensión 12/12, embedding AIM, *HybridCustomTTN*, cabeza lineal, *learning rate* 0,00025, warmup coseno de 5 épocas, label smoothing 0,05, regularización L_2 suave, preservación de relación de aspecto y aumento de datos *light*.

Métrica	Valor
Accuracy top-1	96,05 %
Accuracy top-5	99,43 %
Macro precision	96,12 %
Macro recall	95,82 %
Macro F1	95,94 %
Weighted F1	96,05 %
<code>classification_mAP</code>	98,41 %
Eval loss final	0,2037

Tabla 5.11: Métricas principales del mejor modelo sobre ChemEq25.

Las matrices de confusión ([Figura 5.14](#)) muestran una diagonal muy marcada. Esto significa que el modelo clasifica correctamente la gran mayoría de los recortes. Los errores que quedan se concentran en clases visualmente parecidas. En el mejor experimento, las clases con F1 más bajo siguieron siendo algunas de las más difíciles del dataset, especialmente las relacionadas con objetos alargados o con geometría muy parecida.

El top-5 de 99,43 % también es importante. Quiere decir que, cuando el modelo no acierta la primera predicción, casi siempre tiene la clase correcta entre las cinco más probables. Esto sugiere que la representación aprendida por AIM + TTN contiene mucha información útil, aunque todavía haya confusiones finas entre clases muy similares.

La métrica `classification_mAP` fue 98,41 %. Esta métrica se calculó como una media de precisión one-vs-rest sobre las probabilidades de clase. No debe confundirse con el *mAP@50* de detección del paper de ChemEq25, porque aquí no se evalúan cajas ni IoU. Aun así, es una señal muy fuerte de que el ranking de probabilidades del clasificador es muy bueno.

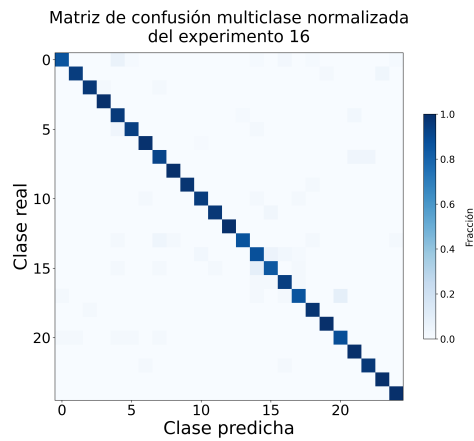
5.7 Variabilidad por seed y límite de las pruebas finales

Después de alcanzar el 96,05 %, se lanzó una *seed sweep* con la misma configuración para comprobar la robustez del modelo.

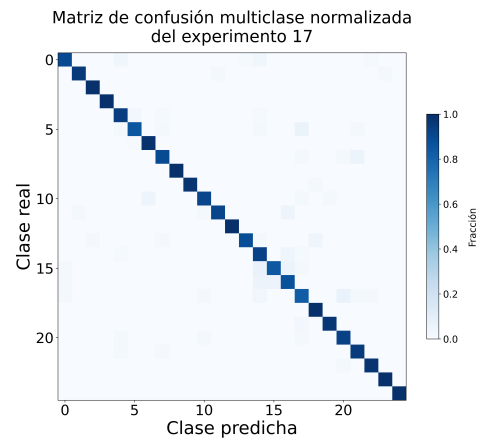
Experimento	Seed	Mejor acc.	Top-5	Macro F1	Acc. final
20	17	92,96 %	98,42 %	92,88 %	91,89 %
21	88	93,40 %	98,78 %	93,41 %	92,39 %
22	123	92,25 %	98,56 %	92,33 %	91,17 %
23	314	93,54 %	98,49 %	93,57 %	93,18 %
Media	–	93,04 %	–	93,05 %	–

Tabla 5.12: Seed sweep posterior al mejor resultado.

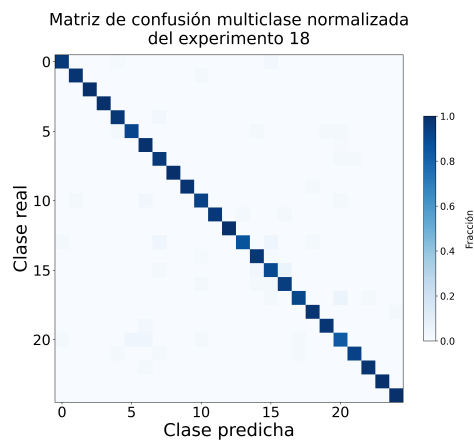
Ninguno de los experimentos superó a `ChemEq_exp15`. Todas quedaron entre 92,25 % y 93,54 %, por debajo del mejor experimento del 14 de mayo. Esto muestra que la *seed* tiene



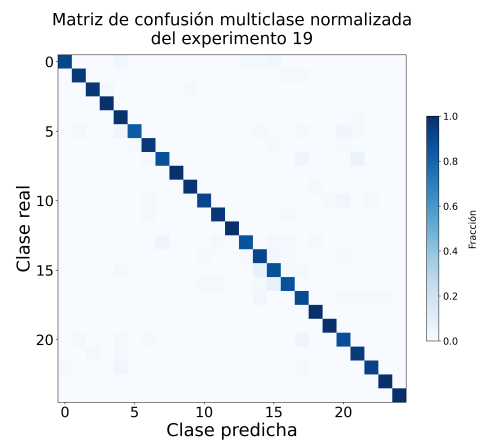
(a)



(b)



(c)



(d)

Figura 5.14: Matrices de confusión de los experimentos después de aplicar *data augmentation*

un efecto fuerte.

También se probó aumentar la capacidad a `local_dim=14` y `bond_dim=14`, pero en esa prueba el resultado cayó a 88,16 %. Esto refuerza una idea importante: en este proyecto, más capacidad no siempre significa mejor generalización. Los cambios que más ayudaron fueron los que mejoraron la información visual y la regularización, no simplemente aumentar dimensiones.

5.8 Resumen de la evolución experimental

La evolución completa del proyecto se resume en la siguiente tabla:

Fase	Cambio principal	Mejor accuracy
Flowers, CustomTTN inicial	TTN propia sin transformaciones intermedias	3,58 %
Flowers, HybridCustomTTN	Transformaciones no lineales entre niveles	13,60 %
Flowers, AIM + augmentation	Front-end AIM y aumento de datos	33,96 %
ChemEq25 inicial	Cambio al dataset principal por crops	57,00 %
ChemEq25 con LR menor	Optimización más estable	75,23 %
ChemEq25 ajustado	Warmup, 12/12, regularización y label smoothing	88,94 %
ChemEq25 final	<i>Data augmentation light</i> y mejor seed	96,05 %

Tabla 5.13: Resumen global de la evolución experimental del proyecto.

La lectura final es que el modelo basado en TN sí consiguió aprender una tarea visual real. El mejor resultado, 96,05 % de accuracy y 95,94 % de macro F1 en ChemEq25, demuestra que la arquitectura AIM + HybridCustomTTN es una alternativa viable para clasificación de imágenes de material de laboratorio.

Capítulo 6. CONCLUSIONES

6.1 Conclusiones del trabajo

El objetivo principal de este Trabajo de Fin de Grado era estudiar la aplicación de las *Tensor Networks* en modelos de *deep learning* para clasificación de imágenes. A partir del trabajo realizado, se puede concluir que este objetivo se ha cumplido de forma satisfactoria. El proyecto no se ha limitado a una revisión teórica de las redes tensoriales, sino que ha incluido el diseño, implementación, entrenamiento y evaluación de un modelo tensorial integrado en un *pipeline* completo de clasificación.

En la parte teórica, el trabajo ha permitido establecer una base sólida sobre los conceptos necesarios para comprender este tipo de modelos, entre ellos los tensores, las contracciones, las descomposiciones tensoriales, la notación diagramática y topologías como MPS, TTN, PEPS o MERA. Esta fundamentación ha resultado clave para justificar las decisiones tomadas más adelante en la implementación, en especial la elección de una arquitectura jerárquica tipo TTN como núcleo del modelo desarrollado.

En el terreno computacional, se ha conseguido construir un sistema entrenable y reproducible basado en PyTorch y TensorKrowch. El proyecto abarca la carga y preparación de datos, el preprocesamiento de imágenes, la generación de recortes a partir de anotaciones YOLO, la definición de experimentos mediante archivos de configuración, el entrenamiento con GPU, el registro de métricas y la generación de diagnósticos. Esta organización ha permitido comparar configuraciones de manera ordenada y seguir la evolución del modelo a lo largo del desarrollo.

Una conclusión relevante es que las *Tensor Networks*, concretamente para clasificación de imágenes, necesitan tener un buen tratamiento de datos e información, para que cuando entren en el modelo, le resulte más fácil diferenciar entre clases y mejorar las predicciones. Los primeros experimentos mostraron que una arquitectura tensorial aplicada directamente sobre representaciones simples tenía dificultades para generalizar. La incorporación de `AIMPatchEmbedding` permitió conservar mejor la información espacial de la imagen antes de entregarla a la TTN, y se convirtió en uno de los cambios clave para mejorar el comportamiento del entrenamiento.

El cambio al *dataset* ChemEq25 también fue decisivo. A diferencia de Flowers102, que presentaba un número elevado de clases visualmente muy parecidas, ChemEq25 permitió abordar un problema más alineado con los objetivos del proyecto, la clasificación de material de laboratorio a partir de recortes individuales.

Los resultados experimentales muestran una mejora progresiva y coherente. El modelo pasó de unos resultados iniciales limitados a alcanzar un rendimiento elevado tras in-

introducir mejoras en la representación de entrada, la optimización, la regularización y el *data augmentation*. El mejor modelo, basado en la combinación de `AIMPatchEmbedding` y `HybridCustomTTN`, obtuvo un 96,05 % de *accuracy*, un 95,94 % de F1 macro y un 99,43 % de *top-5 accuracy* sobre ChemEq25. Estos valores indican que la arquitectura propuesta es capaz de aprender patrones visuales relevantes.

También se ha comprobado que aumentar la capacidad del modelo no garantiza por sí solo una mejor generalización. En los experimentos finales, los cambios que más contribuyeron al rendimiento fueron la preservación de la relación de aspecto, el *label smoothing*, la regularización y, sobre todo, una estrategia suave de *data augmentation*. Esto sugiere que, en modelos tensoriales aplicados a imágenes, la preparación de los datos y el control del entrenamiento pueden llegar a ser tan importantes como la propia topología de la red.

Entre las limitaciones, conviene señalar que no se ha realizado una comparación exhaustiva frente a arquitecturas convencionales como las CNN o los Transformers, ya que el objetivo principal era estudiar la viabilidad de las redes tensoriales, una línea que podría desarrollarse en trabajos posteriores. Asimismo, se ha observado cierta sensibilidad a la *seed* y a la configuración de entrenamiento, lo que indica que estos modelos requieren una validación experimental cuidadosa.

En conjunto, el trabajo demuestra que las *Tensor Networks* pueden integrarse de manera efectiva en modelos de *deep learning* para clasificación de imágenes. Su aplicación práctica exige combinar la estructura tensorial con técnicas habituales del *deep learning*, una representación visual adecuada y un seguimiento riguroso de los experimentos. La principal aportación del proyecto es, por tanto, mostrar un recorrido completo que va desde los fundamentos físicos y matemáticos de las redes tensoriales hasta un modelo funcional, entrenable y evaluable sobre datos reales.

6.2 Conclusiones personales

El desarrollo de este Trabajo de Fin de Grado ha supuesto una experiencia especialmente enriquecedora, tanto desde el punto de vista académico como personal. En primer lugar, el tema abordado pertenece a un ámbito muy especializado, las *tensor networks*, cuyo funcionamiento requiere una comprensión profunda tanto de conceptos físicos como de herramientas computacionales avanzadas. Haber tenido la oportunidad de trabajar en este contexto, y de aprender bajo la supervisión de investigadores con experiencia en el área, en particular Alejandro Mata Ali y todo el departamento de computación cuántica del Instituto Tecnológico de Castilla y León, ha sido un privilegio y una parte fundamental del proceso formativo asociado a este trabajo.

A lo largo de estos meses, este TFG no solo me ha permitido profundizar en la física de las *tensor networks*, sino también adquirir una gran cantidad de competencias técnicas que han resultado esenciales para el desarrollo del proyecto. Entre ellas, destaca el aprendizaje de Python a un nivel mucho más avanzado, así como la experiencia de programar desde

cero un proyecto de cierta envergadura. Esto ha requerido no solo implementar código funcional, sino también aprender a organizarlo correctamente, aplicar buenas prácticas de programación y estructurar el trabajo de forma clara y mantenible.

Además, el proyecto ha supuesto un reto importante en el ámbito del *machine learning*. Ha sido necesario familiarizarse con PyTorch, estudiar las bases matemáticas del *deep learning* y comprender el flujo general de trabajo asociado a este tipo de modelos, desde la preparación de los datos hasta el entrenamiento y análisis de resultados. Todo ello ha ampliado considerablemente mi formación computacional y me ha permitido conectar herramientas modernas de inteligencia artificial con problemas propios de la física teórica y computacional.

Otro aspecto fundamental del desarrollo de este trabajo ha sido el aprendizaje de herramientas habituales en el entorno investigador. Durante el proyecto he aprendido a utilizar Git y GitHub para el control de versiones y la gestión del código, así como comandos básicos de Linux y Bash necesarios para trabajar de manera eficiente en los servidores de la universidad (Lorca). También he tenido la oportunidad de utilizar recursos de computación de alto rendimiento, que han sido clave para llevar a cabo los cálculos y entrenamientos necesarios. Esta experiencia me ha permitido comprender mejor cómo se organiza y ejecuta un proyecto computacional real en un entorno de investigación.

Más allá de los conocimientos técnicos, este TFG me ha acercado al funcionamiento cotidiano de la investigación científica. Haber podido participar en reuniones, observar la forma de trabajo del grupo y ver de primera mano cómo se plantean, discuten y desarrollan problemas de investigación ha sido una parte especialmente valiosa de la experiencia. En este sentido, el trabajo no solo ha consistido en resolver un problema concreto, sino también en aprender cómo se investiga, cómo se colabora y cómo se construye conocimiento en un contexto académico real.

En conjunto, este TFG ha representado un desafío constante. Por un lado, debido a la dificultad conceptual asociada a la física de las *tensor networks*; por otro, por la exigencia técnica derivada de la programación, el uso de herramientas de aprendizaje automático y el trabajo en entornos computacionales avanzados. Sin embargo, precisamente esta combinación de retos ha hecho que el proceso sea especialmente formativo. En apenas unos meses he adquirido conocimientos y habilidades que al inicio del proyecto no imaginaba poder desarrollar, y considero que esta experiencia ha supuesto un punto de inflexión importante en mi formación como físico.

En definitiva, ha sido un proyecto muy ambicioso que ha puesto a prueba todos mis conocimientos sobre física aprendidos en la carrera y ha ampliado considerablemente esos conocimientos, mas especializados en la física computacional. También me ha permitido crecer a nivel científico, ya que he aprendido muchas herramientas que me servirán en mi carrera profesional.

REFERENCIAS

- [1] Firefly meta-heuristic algorithm applied to artificial neural network pre-training. Scientific Figure on ResearchGate.
- [2] Fred Van Veen and Stefan Leijnen. The neural network zoo. <https://www.asimovinstitute.org/neural-network-zoo>, 2019. Asimov Institute. Accessed: 2026-03-19.
- [3] Alejandro Mata Ali. Tensor Network Editor, May 2026.
- [4] Alejandro Mata Ali, Iñigo Perez Delgado, and Aitor Moreno Fdez. de Leceta. Quantum-inspired Techniques in Tensor Networks for Industrial Contexts. *arXiv e-prints*, page arXiv:2404.11277, April 2024.
- [5] Maolin Wang, Yu Pan, Zenglin Xu, Guangxi Li, Xiangli Yang, Danilo Mandic, and Andrzej Cichocki. Tensor networks meet neural networks: A survey and future perspectives. *arXiv preprint arXiv:2302.09019*, 2025. Version 3, revised 17 Mar 2025.
- [6] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P Mandic. Tensor networks for dimensionality reduction and large-scale optimization part 1 low-rank tensor decompositions. *Foundations and Trends in Machine Learning*, 9(4-5):249–429, 2016.
- [7] E. Miles Stoudenmire and David J. Schwab. Supervised learning with quantum-inspired tensor networks, 2017.
- [8] Song Cheng, Lei Wang, and Pan Zhang. Supervised learning with projected entangled pair states. *Physical Review B*, 103(12):125117, 2021.
- [9] Norbert Schuch, Michael M. Wolf, Frank Verstraete, and J. Ignacio Cirac. Computational complexity of projected entangled pair states. *Physical Review Letters*, 98(14):140506, 2007.
- [10] G. Evenbly and G. Vidal. Tensor Network States and Geometry. *Journal of Statistical Physics*, 145(4):891–918, November 2011.
- [11] Chang Nie, Junfang Chen, and Yajie Chen. Deep tree tensor networks for image recognition, 2025. NeurIPS 2025 poster; arXiv:2502.09928.
- [12] José Ramón Pareja Monturiol, David Pérez-García, and Alejandro Pozas-Kerstjens. Tensorkrowch: Smooth integration of tensor networks in machine learning. *Quantum*, 8:1364, 2024.

- [13] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.
- [14] Roger Penrose et al. Applications of negative dimensional tensors. *Combinatorial mathematics and its applications*, 1(221-244):3, 1971.
- [15] Jacob Biamonte and Ville Bergholm. Tensor networks in a nutshell, 2017.
- [16] Ulrich Schollwoeck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics - ANN PHYS N Y*, 326, 08 2010.
- [17] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- [18] Ivan Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33:2295–2317, 09 2011.
- [19] G. Vidal. Entanglement renormalization. *Phys. Rev. Lett.*, 99:220405, Nov 2007.
- [20] F. Verstraete and J. I. Cirac. Renormalization algorithms for quantum-many body systems in two and higher dimensions. *arXiv preprint cond-mat/0407066*, 2004.
- [21] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, Dec 2008.
- [22] Md Sakhawat Hossain et al. Real-world chemistry lab image dataset for equipment recognition across 25 apparatus categories. *Scientific Data*, 12(1747), 2025.

[PÁGINA INTENCIONADAMENTE EN BLANCO]